

**ĐẠI HỌC QUỐC GIA HÀ NỘI  
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



**Phan Tiến Đạt**

**ARGOS: ORCHESTRATING GRAPH  
REASONING WITH ADAPTIVE AGENTS FOR  
ROBUST CONVERSATIONAL  
RECOMMENDATION**

**KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY**

**Ngành: Hệ thống Thông tin**

**HÀ NỘI - 2026**

ĐẠI HỌC QUỐC GIA HÀ NỘI  
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

Phan Tiến Đạt

**ARGOS: ORCHESTRATING GRAPH  
REASONING WITH ADAPTIVE AGENTS FOR  
ROBUST CONVERSATIONAL  
RECOMMENDATION**

**KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY**

**Ngành: Hệ thống Thông tin**

**Cán bộ hướng dẫn: TS. Nguyễn Thị Hậu**

**HÀ NỘI - 2026**

**VIETNAM NATIONAL UNIVERSITY, HANOI  
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**

**Phan Tien Dat**

**ARGOS: ORCHESTRATING GRAPH  
REASONING WITH ADAPTIVE AGENTS FOR  
ROBUST CONVERSATIONAL  
RECOMMENDATION**

**Major: Information System**

**Supervisor: Dr. Nguyen Thi Hau**

**HANOI - 2026**

# Lời cảm ơn

Trước hết, tôi xin bày tỏ lòng biết ơn sâu sắc đến TS. Nguyễn Thị Hậu vì sự hỗ trợ kiên định, sự dìu dắt vô giá và những định hướng khoa học sâu sắc trong suốt quá trình tôi thực hiện khóa luận này. Sự tận tâm và chuyên môn của cô không chỉ là kim chỉ nam giúp tôi hoàn thiện công trình nghiên cứu, mà còn đóng vai trò nền tảng trong sự phát triển tư duy cá nhân của tôi. Nếu không có sự động viên không ngừng và sự dẫn dắt chu đáo của cô, nghiên cứu này đã không thể trở thành hiện thực.

Bên cạnh đó, tôi xin được bày tỏ lòng biết ơn sâu sắc đến toàn thể các thầy cô tại Trường Đại học Công nghệ - Đại học Quốc gia Hà Nội. Trong suốt 4 năm học tập tại trường, tôi đã được nuôi dưỡng trong một môi trường giáo dục chuyên nghiệp, đầy cảm hứng và nhân văn. Sự tận tâm trong giảng dạy cùng những khối kiến thức mà các thầy cô truyền đạt đã giúp tôi xây dựng một nền tảng tri thức vững chắc.

Cuối cùng, tôi xin trân trọng cảm ơn gia đình và tất cả những người thân yêu đã luôn đồng hành, là chỗ dựa tinh thần vững chắc và tạo mọi điều kiện tốt nhất để tôi có thể hoàn thành khóa luận tốt nghiệp này.

# Lời cam đoan

Tôi xin cam đoan rằng luận văn này là kết quả nghiên cứu của chính bản thân tôi, được thực hiện dưới sự hướng dẫn khoa học của các thầy cô. Toàn bộ nội dung, số liệu, kết quả và các phân tích trong luận văn đều do tôi trực tiếp thực hiện, hoặc được tham khảo từ những nguồn tài liệu khoa học đáng tin cậy và đã được trích dẫn, ghi rõ nguồn theo đúng quy định.

Tôi khẳng định rằng luận văn này không sao chép của bất kỳ công trình nào khác, không vi phạm bản quyền của cá nhân hay tổ chức nào. Mọi ý tưởng, phương pháp, hay tài liệu sử dụng từ các công trình nghiên cứu của người khác đều đã được tôi trích dẫn và ghi nhận đầy đủ.

Tôi cam kết rằng tất cả những tuyên bố trên đều là sự thật. Tôi xin hoàn toàn chịu trách nhiệm trước pháp luật cũng như các quy định của Trường Đại học Công nghệ - Đại học Quốc gia Hà Nội về những cam đoan trên.

Hà Nội, ngày ... tháng ... năm 2026

Sinh viên

Phan Tiến Đạt

# Mục lục

|                                                                              |             |
|------------------------------------------------------------------------------|-------------|
| <b>Lời cảm ơn</b>                                                            | <b>i</b>    |
| <b>Lời cam đoan</b>                                                          | <b>ii</b>   |
| <b>Mục lục</b>                                                               | <b>iii</b>  |
| <b>Tóm tắt</b>                                                               | <b>vii</b>  |
| <b>Danh sách hình vẽ</b>                                                     | <b>viii</b> |
| <b>Danh sách bảng</b>                                                        | <b>xi</b>   |
| <b>Thuật ngữ</b>                                                             | <b>xii</b>  |
| <b>Mở đầu</b>                                                                | <b>1</b>    |
| <b>Chương 1 Giới thiệu chung</b>                                             | <b>4</b>    |
| 1.1 Đặt vấn đề và Động lực nghiên cứu . . . . .                              | 4           |
| 1.2 Tổng quan các nghiên cứu liên quan . . . . .                             | 5           |
| 1.2.1 Hệ thống CRS truyền thống tăng cường tri thức . . . . .                | 5           |
| 1.2.2 Mô hình Ngôn ngữ Lớn trong Hệ thống gợi ý . . . . .                    | 6           |
| 1.2.3 Các kiến trúc lai và tăng cường truy xuất (RAG) . . . . .              | 7           |
| 1.3 Khoảng trống nghiên cứu và Sự cần thiết của hệ thống đa tác tử . . . . . | 7           |
| 1.4 Mục tiêu nghiên cứu . . . . .                                            | 9           |
| 1.5 Đóng góp của khóa luận . . . . .                                         | 10          |
| <b>Chương 2 Kiến thức nền tảng</b>                                           | <b>12</b>   |
| 2.1 Định nghĩa bài toán Hệ thống Gợi ý Hội thoại (CRS) . . . . .             | 12          |

|                                                             |                                                                                            |           |
|-------------------------------------------------------------|--------------------------------------------------------------------------------------------|-----------|
| 2.2                                                         | Đồ thị Tri thức (Knowledge Graph - KG) . . . . .                                           | 13        |
| 2.3                                                         | Mô hình Ngôn ngữ Lớn (LLMs) và Kỹ thuật RAG . . . . .                                      | 15        |
| 2.3.1                                                       | Mô hình Ngôn ngữ Lớn (LLMs) . . . . .                                                      | 15        |
| 2.3.2                                                       | Kỹ thuật RAG (Retrieval-Augmented Generation) . . . . .                                    | 16        |
| 2.4                                                         | Kiến trúc Hệ thống Đa tác tử (Multi-Agent System) và ReAct . . . . .                       | 17        |
| 2.4.1                                                       | Hệ thống Đa tác tử dựa trên LLM . . . . .                                                  | 17        |
| 2.4.2                                                       | Vòng lặp ReAct (Reasoning and Acting) . . . . .                                            | 17        |
| 2.5                                                         | Bộ dữ liệu thực nghiệm . . . . .                                                           | 19        |
| 2.6                                                         | Các phương pháp đánh giá hệ thống (Evaluation Metrics) . . . . .                           | 19        |
| <b>Chương 3 Nghiên cứu tiền đề: Hệ thống GRACE</b>          |                                                                                            | <b>21</b> |
| 3.1                                                         | Tổng quan kiến trúc GRACE . . . . .                                                        | 21        |
| 3.2                                                         | Xây dựng Đồ thị tri thức (Knowledge Graph Construction) . . . . .                          | 22        |
| 3.3                                                         | Tóm tắt hội thoại và xây dựng Hồ sơ sở thích (Dialogue Summarization) . . . . .            | 23        |
| 3.4                                                         | Module Truy xuất Lai (Hybrid Retriever) . . . . .                                          | 24        |
| 3.4.1                                                       | Truy xuất ngữ nghĩa (Semantic Search) . . . . .                                            | 24        |
| 3.4.2                                                       | Lọc theo nội dung (Content-based Filtering) . . . . .                                      | 25        |
| 3.4.3                                                       | Mở rộng đồ thị (Graph Expansion - Collaborative) . . . . .                                 | 26        |
| 3.4.4                                                       | Cơ chế hợp nhất (Hybrid Integration) . . . . .                                             | 27        |
| 3.5                                                         | Module Xếp hạng lại bằng Generative LLM . . . . .                                          | 27        |
| 3.6                                                         | Phân tích điểm nghẽn mang tính cấu trúc . . . . .                                          | 28        |
| <b>Chương 4 Kiến trúc đề xuất: Hệ thống Đa tác tử ARGOS</b> |                                                                                            | <b>30</b> |
| 4.1                                                         | Động lực chuyển đổi sang Vòng lặp phản hồi Đa tác tử (Multi-Agent Feedback Loop) . . . . . | 30        |
| 4.2                                                         | Tác tử Hồ sơ (Profiler Agent) và Phân rã ý định . . . . .                                  | 31        |
| 4.2.1                                                       | Trích xuất thực thể: Ràng buộc cứng và Hạt giống gợi ý . . . . .                           | 33        |
| 4.2.2                                                       | Đa dạng hóa ngữ nghĩa (Intent Expansion / Multi-Query) . . . . .                           | 34        |
| 4.2.3                                                       | Dự báo chiến lược trọng số động (Dynamic Weighting / Adaptive Routing) . . . . .           | 35        |

|                 |                                                                                             |           |
|-----------------|---------------------------------------------------------------------------------------------|-----------|
| 4.3             | Tác tử Đồ thị (Graph Agent) với Vòng lặp ReAct . . . . .                                    | 37        |
| 4.3.1           | Nhận thức cấu trúc dữ liệu (Schema-Aware Intelligence) . . . . .                            | 37        |
| 4.3.2           | Vòng lặp Suy luận và Hành động (Reasoning + Acting) . . . . .                               | 38        |
| 4.4             | Trung tâm Điều phối (Orchestrator Agent) . . . . .                                          | 40        |
| 4.4.1           | Thực thi truy xuất bất đồng bộ (Asynchronous Parallel Execution) . . .                      | 41        |
| 4.4.2           | Hợp nhất thứ hạng với thuật toán WRRF (Dynamic WRRF) . . . . .                              | 41        |
| 4.5             | Tác tử Kiểm định (Critic Agent) - Kiểm soát chất lượng . . . . .                            | 42        |
| 4.5.1           | Đối soát chéo và triệt tiêu ảo giác (Anti-Hallucination) . . . . .                          | 42        |
| 4.5.2           | Kích hoạt phản hồi và Lập báo cáo lỗi (Feedback Trigger & Error Re-<br>reporting) . . . . . | 44        |
| 4.6             | Tác tử Thỏa hiệp (Relaxation Agent) - Cơ chế tự sửa lỗi . . . . .                           | 45        |
| 4.6.1           | Phân tích điểm nghẽn và Phân cấp ưu tiên (Sacrifice Hierarchy) . . . .                      | 45        |
| 4.6.2           | Tái cấu trúc ý định (Intent Reconstruction) . . . . .                                       | 47        |
| 4.7             | Cải tiến Module Xếp hạng hai giai đoạn (Decoupled Reranking) . . . . .                      | 48        |
| 4.8             | Sinh phản hồi hội thoại (Dialogue Response Generation) . . . . .                            | 49        |
| <b>Chương 5</b> | <b>Thiết lập và Kết quả Thực nghiệm</b>                                                     | <b>51</b> |
| 5.1             | Thiết lập thí nghiệm . . . . .                                                              | 51        |
| 5.1.1           | Tập dữ liệu ReDial và INSPIRED . . . . .                                                    | 51        |
| 5.1.2           | Cấu hình hệ thống và Phương pháp đánh giá . . . . .                                         | 52        |
| 5.2             | Kết quả đánh giá hệ thống . . . . .                                                         | 53        |
| 5.2.1           | Khẳng định năng lực của kiến trúc tiền đề (GRACE vs. SOTA) . . . .                          | 53        |
| 5.2.2           | Đánh giá so sánh nội bộ: GRACE (Static) và ARGOS (Đa tác tử) . . .                          | 54        |
| 5.3             | Nghiên cứu tình huống (Case Studies) . . . . .                                              | 56        |
| 5.3.1           | Đa truy vấn: Khám phá các mối liên hệ ẩn (Hidden Relational Gems) .                         | 56        |
| 5.3.2           | Tác tử Đồ thị (Graph Agent): Nội suy thông tin từ dữ liệu khuyết thiếu .                    | 56        |
| 5.3.3           | Tác tử Kiểm định (Critic Agent): Kiểm định ràng buộc xuyên luồng . .                        | 57        |
| 5.3.4           | Tác tử Thỏa hiệp (Relaxation Agent): Điều chỉnh ý định truy vấn . . .                       | 57        |



|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>Kết luận</b>                                           | <b>59</b> |
| 5.4 Tổng kết các đóng góp . . . . .                       | 59        |
| 5.5 Hạn chế còn tồn tại . . . . .                         | 60        |
| 5.6 Hướng phát triển tương lai . . . . .                  | 61        |
| 5.7 Mối quan hệ với công trình tiền đề GRACE . . . . .    | 61        |
| 5.8 Danh mục các công trình khoa học đã công bố . . . . . | 62        |
| <b>Danh mục tham khảo</b>                                 | <b>63</b> |

# Tóm tắt

Hệ thống gợi ý hội thoại (Conversational Recommender System - CRS) đóng vai trò trung tâm trong việc giải quyết bài toán thông tin cá nhân hóa thông qua tương tác ngôn ngữ tự nhiên. Tuy nhiên, sự kết hợp giữa các Mô hình ngôn ngữ lớn (LLM) và các kho tri thức cấu trúc vẫn tồn tại những rào cản về tính xác thực của dữ liệu và năng lực suy luận linh hoạt trong các kịch bản thực tế.

Khóa luận này đề xuất hệ thống **ARGOS** (Adaptive Reasoning and Graph Orchestration System), một khung làm việc đa tác tử được kế thừa và phát triển từ mô hình tiền đề **GRACE** (công trình nghiên cứu của chính tác giả đã được công bố tại hội nghị quốc tế SOICT 2025). Khác với các kiến trúc dòng chảy tĩnh vốn dễ gặp bế tắc khi đối mặt với các yêu cầu phức tạp, ARGOS vận hành dựa trên cơ chế cộng tác giữa các tác tử chuyên biệt: Profiler Agent (Phân tích hồ sơ), Orchestrator (Điều phối), Graph Agent (Suy luận đồ thị), Critic Agent (Kiểm định) và Relaxation Agent (Thỏa hiệp). Kiến trúc này cho phép hệ thống thực thi các vòng lặp phản hồi để tự phát hiện ảo giác, đồng thời tự động điều chỉnh chiến lược truy xuất khi gặp phải các ràng buộc mâu thuẫn từ phía người dùng.

Kết quả đánh giá thực nghiệm trên các bộ dữ liệu tiêu chuẩn ReDial và INSPIRED chứng minh rằng ARGOS không chỉ bảo toàn tính chính xác tuyệt đối nhờ sự ràng buộc vào Đồ thị tri thức, mà còn cải thiện đáng kể khả năng hiểu ý định và tính linh hoạt trong hội thoại so với kiến trúc tiền đề. Nghiên cứu này khẳng định tính hiệu quả của phương pháp tiếp cận đa tác tử trong việc xây dựng các hệ thống gợi ý hội thoại thông minh, tin cậy và có khả năng mở rộng.

**Từ khóa:** Hệ thống gợi ý hội thoại, Đồ thị tri thức, Mô hình ngôn ngữ lớn, Hệ thống đa tác tử (Multi-Agent System), GRACE, ARGOS.

# Danh sách hình vẽ

|     |                                                                                                                                                                                                                                                                                      |    |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Ví dụ hội thoại nhiều lượt trong Hệ thống gợi ý hội thoại. Hệ thống chủ động đặt câu hỏi làm rõ sở thích, cập nhật trạng thái hội thoại và đưa ra gợi ý kèm giải thích qua từng lượt tương tác [1]. . . . .                                                                          | 14 |
| 2.2 | Ví dụ cấu trúc một node Phim trong Đồ thị tri thức, liên kết với các thực thể Đạo diễn, Diễn viên, Thể loại và các thuộc tính định lượng. Mỗi bộ ba $(h, r, t)$ biểu diễn một quan hệ có hướng giữa hai thực thể trong đồ thị.                                                       | 15 |
| 2.3 | Pipeline RAG cơ bản gồm ba giai đoạn: <i>Retrieval</i> (truy xuất tài liệu liên quan từ kho dữ liệu ngoài), <i>Augmentation</i> (tích hợp tài liệu vào ngữ cảnh) và <i>Generation</i> (LLM sinh câu trả lời được neo giữ vào dữ liệu thực chứng) [2]. . . . .                        | 16 |
| 2.4 | So sánh các phương pháp prompting LLM: Standard, Chain-of-Thought, Act-only và <b>ReAct</b> . ReAct xen kẽ giữa suy luận (Thought) và hành động tương tác với môi trường (Action-Observation), cho phép tác tử tự điều chỉnh chiến lược dựa trên phản hồi từ môi trường [3]. . . . . | 18 |
| 3.1 | Tổng quan kiến trúc hệ thống tiền đề GRACE, bao gồm hai module chính: LLM Retriever và LLM Re-ranker. . . . .                                                                                                                                                                        | 22 |
| 3.2 | Cấu trúc một node Phim (Film) trong Đồ thị tri thức. Node này liên kết trực tiếp với các thuộc tính (Năm, Điểm số, Thể loại) và các thực thể sáng tạo (Đạo diễn, Diễn viên). . . . .                                                                                                 | 23 |
| 3.3 | Quy trình truy xuất ngữ nghĩa dựa trên phép tính tương đồng Cosine giữa vector của câu truy vấn và vector cốt truyện phim. . . . .                                                                                                                                                   | 25 |

|     |                                                                                                                                                                                                                                                                         |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.4 | Cơ chế lọc theo nội dung dựa trên các thuộc tính cứng (Thể loại, Năm sản xuất, Điểm số). . . . .                                                                                                                                                                        | 26 |
| 3.5 | Thuật toán mở rộng đồ thị dựa trên việc chia sẻ các node đội ngũ sáng tạo (Diễn viên, Đạo diễn). . . . .                                                                                                                                                                | 26 |
| 3.6 | Kiến trúc Module Xếp hạng lại, sử dụng cơ chế batching để xử lý tập ứng viên lớn thông qua LLM. . . . .                                                                                                                                                                 | 28 |
| 4.1 | Kiến trúc tổng quan của hệ thống đa tác tử ARGOS. Khác với kiến trúc tĩnh, ARGOS đưa vào các vòng lặp phản hồi nội bộ (feedback loops) và cơ chế đánh giá lại trọng số động thông qua các Agent chuyên biệt. . . . .                                                    | 31 |
| 4.2 | Luồng suy luận của Profiler Agent: phân loại thông tin đầu vào thành ba nhánh độc lập (Explicit, Abstract, Behavioral) và tổng hợp bộ trọng số WRRF. . . . .                                                                                                            | 32 |
| 4.3 | Cơ chế dự báo trọng số động (Dynamic Weight Prediction) do Profiler Agent thực hiện, cung cấp tín hiệu định tuyến cho các luồng truy xuất. . . . .                                                                                                                      | 36 |
| 4.4 | Chu trình ReAct của Graph Agent: xen kẽ Suy luận - Hành động - Quan sát để tự sửa lỗi và tối ưu chiến lược truy vấn đồ thị. . . . .                                                                                                                                     | 39 |
| 4.5 | Chiến lược tìm kiếm đa luồng của Orchestrator Agent. Hệ thống kích hoạt song song ba luồng độc lập: Vòng lặp ReAct của Graph Agent, Semantic Retrieval và Content Retrieval, sau đó hợp nhất kết quả thông qua thuật toán WRRF Fusion có sử dụng trọng số động. . . . . | 40 |
| 4.6 | Cơ chế hoạt động của Critic Agent. Tác tử thực hiện đối soát chéo các ứng viên với Ràng buộc cứng (Hard Constraints) và kích hoạt cơ chế phản hồi (Feedback Trigger) nếu số lượng ứng viên hợp lệ không đạt ngưỡng dung sai. . . . .                                    | 43 |
| 4.7 | Quy trình phân tích điểm nghẽn và tái cấu trúc ý định của Relaxation Agent. Dựa trên phân cấp ưu tiên (Sacrifice Hierarchy), tác tử nới lỏng các điều kiện tìm kiếm và khởi động lại vòng lặp hội thoại mới. . . . .                                                    | 46 |

|     |                                                                                                                                                                                                                   |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.8 | Quy trình xếp hạng hai giai đoạn (2-stage Reranking). Hệ thống kết hợp mô hình Rerank chuyên dụng để lọc nhanh danh sách ứng viên, sau đó tận dụng LLM để đánh giá sâu logic và đưa ra kết quả cuối cùng. . . . . | 49 |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|

# Danh sách bảng

|     |                                                                                                                                                                          |    |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Tổng hợp các nghiên cứu liên quan trong lĩnh vực Hệ thống gợi ý hội thoại.                                                                                               | 8  |
| 5.1 | Siêu tham số hệ thống ARGOS . . . . .                                                                                                                                    | 52 |
| 5.2 | Hiệu năng của Mô hình tiền đề (GRACE) so với các hệ thống SOTA trên toàn bộ tập dữ liệu (100%). Các kết quả tốt nhất được in đậm, và tốt thứ hai được gạch chân. . . . . | 54 |
| 5.3 | Đánh giá nội bộ: So sánh hiệu năng giữa GRACE (Dòng chảy tĩnh) và ARGOS (Hệ thống Đa tác tử) trên tập mẫu 20% có tính ý nghĩa thống kê ( $p < 0.05$ ). . . . .           | 55 |
| 5.4 | Phân định đóng góp giữa bài báo GRACE (SOICT 2025) và khóa luận ARGOS. . . . .                                                                                           | 62 |

# Thuật ngữ

| Từ viết tắt | Thuật ngữ                         | Ý nghĩa                                                       |
|-------------|-----------------------------------|---------------------------------------------------------------|
| CRS         | Conversational Recommender System | Hệ thống gợi ý hội thoại                                      |
| KG          | Knowledge Graph                   | Đồ thị tri thức                                               |
| LLM         | Large Language Model              | Mô hình ngôn ngữ lớn                                          |
| SLM         | Small Language Model              | Mô hình ngôn ngữ kích thước nhỏ                               |
| RAG         | Retrieval-Augmented Generation    | Sinh văn bản tăng cường truy xuất                             |
| ReAct       | Reasoning and Acting              | Mô hình kết hợp Suy luận và Hành động                         |
| WRRF        | Weighted Reciprocal Rank Fusion   | Thuật toán hợp nhất thứ hạng có trọng số                      |
| SOTA        | State-Of-The-Art                  | Các kỹ thuật/mô hình tiên tiến nhất hiện nay                  |
| Multi-Agent | Multi-Agent System                | Hệ thống cấu thành từ nhiều tác tử độc lập                    |
| Recall      | Recall                            | Độ bao phủ (Tỷ lệ tìm thấy các đối tượng phù hợp)             |
| Precision   | Precision                         | Độ chính xác (Tỷ lệ đối tượng phù hợp trong số được tìm thấy) |
| API         | Application Programming Interface | Giao diện lập trình ứng dụng                                  |
| GDBMS       | Graph Database Management System  | Hệ quản trị cơ sở dữ liệu đồ thị                              |

# Mở đầu

Sự bùng nổ của thông tin số cùng nhu cầu cá nhân hóa ngày càng cao đã thúc đẩy sự chuyển dịch mạnh mẽ từ các hệ thống gợi ý tĩnh truyền thống sang các hệ thống gợi ý hội thoại (Conversational Recommender System - CRS). Bằng cách sử dụng ngôn ngữ tự nhiên làm phương thức tương tác chủ đạo, CRS cho phép hệ thống khai thác các sở thích phức tạp, linh hoạt và xử lý các nhu cầu mơ hồ của người dùng thông qua nhiều lượt đối thoại [4, 5]. Các bài toán lõi của CRS bao gồm việc nắm bắt chính xác ý định người dùng từ ngữ cảnh hội thoại, sinh ra các mục gợi ý phù hợp và tạo ra phản hồi ngôn ngữ tự nhiên để tương tác. Đây là một lĩnh vực thu hút sự quan tâm lớn từ cộng đồng nghiên cứu. Việc cá nhân hóa hội thoại đóng vai trò then chốt trong việc nâng cao trải nghiệm người dùng, giữ chân khách hàng và trực tiếp thúc đẩy giá trị thương mại cho các nền tảng dịch vụ.

Tuy nhiên, việc xây dựng một hệ thống CRS hoàn thiện đưa ra những thách thức đặc biệt. Sự xuất hiện của các Mô hình ngôn ngữ lớn (LLMs) đã cách mạng hóa năng lực xử lý ngôn ngữ, nhưng việc tích hợp LLM vào CRS hiện nay vẫn vấp phải rào cản cốt lõi: Tính tin cậy của thông tin (hiện tượng ảo giác) và giới hạn về năng lực suy luận đa bước. Khác với các hệ thống phân lớp hoặc dự đoán thông thường, hệ thống gợi ý hội thoại đòi hỏi hệ thống phải liên tục đối soát ý định của người dùng với cơ sở dữ liệu thực tế (như Đồ thị tri thức) để đảm bảo không gợi ý những mục không tồn tại.

Trong các nghiên cứu trước đây, nhiều kỹ thuật và kiến trúc khác nhau đã được đề xuất nhằm giải quyết bài toán này. Các hệ thống truyền thống tăng cường tri thức như KBRD [6], KGSF [7] hay KERL [8] đã tận dụng Đồ thị tri thức nhưng lại gặp hạn chế về khả năng hiểu ngôn ngữ linh hoạt. Các phương pháp dựa trên LLM theo hướng Zero-Shot (như LLM-ReDial [9]) tỏ ra linh hoạt nhưng rất dễ sinh ra ảo giác do phụ thuộc hoàn toàn vào trí nhớ tham số. Các phương pháp tinh chỉnh như TallRec [10] hay ReFICR



[11] mang lại độ chính xác cao nhưng tốn kém tài nguyên và thiếu linh hoạt khi cập nhật dữ liệu mới. Gần đây, việc áp dụng kỹ thuật Truy xuất tăng cường tạo sinh (Retrieval-Augmented Generation - RAG) trên Đồ thị tri thức đã cho thấy những kết quả đầy hứa hẹn. Điển hình, kiến trúc tiền đề GRACE (Graph-Reasoning Augmented Conversational Engine) đã giải quyết hiệu quả vấn đề ảo giác thông qua luồng truy xuất lai và sử dụng mô hình ngôn ngữ lớn để tái xếp hạng. Tuy nhiên, GRACE và các hệ thống RAG hiện tại vẫn hoạt động theo kiến trúc đường ống tĩnh: khi đối mặt với các truy vấn mâu thuẫn hoặc không có kết quả khớp tuyệt đối, chúng thiếu cơ chế tự đánh giá, tự sửa lỗi và thỏa hiệp, dẫn đến kết quả rỗng hoặc chất lượng thấp, làm gián đoạn mạch hội thoại.

Để khắc phục hạn chế đó, khóa luận đề xuất hệ thống **ARGOS** (Adaptive Reasoning and Graph Orchestration System) chuyển đổi từ mô hình xử lý tuyến tính sang mô hình đa tác tử. Bằng cách phân chia các chức năng vào các tác tử chuyên biệt và thiết lập vòng lặp phản hồi, khóa luận hướng tới các đóng góp chính như sau:

(1) **Chuyển đổi kiến trúc từ tĩnh sang đa tác tử thích nghi:** Khóa luận đề xuất kiến trúc cộng tác giữa năm tác tử chuyên biệt: Profiler Agent, Orchestrator, Graph Agent, Critic Agent và Relaxation Agent. Khung làm việc này loại bỏ điểm nghẽn trên bằng cách đưa vào các vòng lặp phản hồi nội bộ. Hệ thống có khả năng tự động phân rã ý định người dùng qua Profiler Agent, điều phối truy xuất qua Orchestrator, suy luận đa bước qua Graph Agent, và tự đánh giá lại kết quả qua cặp Critic-Relaxation Agent. Khóa luận xác thực tính hiệu quả của mô hình thông qua hai bộ dữ liệu hội thoại tiêu chuẩn là ReDial và INSPIRED.

(2) **Thiết lập cơ chế kiểm định nghiêm ngặt và triệt tiêu ảo giác:** Khóa luận trình bày một phương pháp đối soát chéo độc lập thông qua tác tử Critic Agent. Bằng cách tách rời khâu trích xuất ràng buộc cứng và khâu kiểm định, mọi ứng viên được sinh ra đều phải trải qua bộ lọc của Critic Agent đối chiếu trực tiếp với Đồ thị tri thức. Phương pháp này loại bỏ các ứng viên vi phạm ràng buộc cứng của người dùng và bổ sung lớp kiểm định còn thiếu, đảm bảo mọi gợi ý đều có cơ sở kiểm chứng từ Đồ thị tri thức.

(3) **Khả năng tự sửa lỗi và thỏa hiệp thông minh:** Khi đối diện với các yêu cầu khắt khe dẫn đến tập kết quả rỗng, Relaxation Agent được kích hoạt. Dựa trên thuật toán phân cấp ưu tiên, hệ thống có khả năng tự động phân tích điểm nghẽn, nới lỏng các ràng buộc ít quan trọng (ví dụ: năm phát hành) và giữ lại các giá trị cốt lõi (ví dụ: phong cách đạo diễn). Khả năng tự khởi động lại luồng truy vấn với ý định đã được tái cấu trúc giúp

hệ thống khôi phục hội thoại một cách tự nhiên mà không cần đến sự can thiệp thủ công từ người dùng.

(4) **Tối ưu hóa độ trễ qua cơ chế Xếp hạng hai giai đoạn:** Nhằm đảm bảo khả năng đáp ứng thời gian thực, hệ thống đề xuất tách rời quá trình xếp hạng. Việc kết hợp mô hình Reranker thương mại nhẹ gọn cho vòng lọc đầu và giới hạn sự can thiệp của LLM lớn ở vòng suy luận cuối giúp giảm thiểu đáng kể chi phí tính toán và độ trễ, trong khi vẫn duy trì được hiệu suất gợi ý vượt trội. Thông qua cơ chế này, hệ thống duy trì hiệu năng gợi ý cạnh tranh mà không yêu cầu tinh chỉnh mô hình.

Phần còn lại của khóa luận được tổ chức như sau:

Chương 1 giới thiệu tổng quan, đặt vấn đề, tổng hợp các nghiên cứu liên quan, xác định khoảng trống nghiên cứu cùng mục tiêu và đóng góp của khóa luận. Chi tiết các kiến thức nền tảng về Hệ thống gợi ý hội thoại, Đồ thị tri thức, Mô hình ngôn ngữ lớn và kiến trúc Đa tác tử được trình bày tại Chương 2. Chương 3 đi sâu phân tích hệ thống tiền đề GRACE, chỉ ra những ưu điểm và định vị các điểm nghẽn của kiến trúc RAG tĩnh. Chi tiết các bước và cơ chế hoạt động của khung làm việc đa tác tử đề xuất ARGOS được mô tả kỹ lưỡng tại Chương 4. Chương 5 thể hiện kịch bản, thiết lập và các kết quả thực nghiệm, bao gồm phân tích chuyên sâu thông qua các chỉ số hiệu suất và so sánh với kết quả của các nghiên cứu hàng đầu hiện nay. Phần Kết luận được trình bày tại Chương 6, tóm lược các thành tựu chính đạt được và đề xuất các hướng phát triển trong tương lai.

Các nghiên cứu phân tích và cấu trúc mô hình tiền đề của khóa luận đã đóng góp vào nền tảng của bài báo khoa học<sup>1</sup> do người viết là tác giả chính [12]. Sự ghi nhận bước đầu về mặt học thuật này khẳng định tính đúng đắn của hướng nghiên cứu, đồng thời là cơ sở thực tiễn quan trọng để người viết tiếp tục phát triển và tối ưu hóa kiến trúc hệ thống đa tác tử trong phạm vi khóa luận này.

---

<sup>1</sup>Thông tin bài báo: Phan Tiến Đạt, Trần Vũ Anh, Hoàng Bảo Long, Trương Thị Minh Ngọc, Nguyễn Thị Hậu. "GRACE: A Knowledge Graph-Enhanced Conversational Recommendation System via Retrieval-Augmented Generation", Hội nghị Quốc tế về Công nghệ Thông tin và Truyền thông lần thứ 14 (SOICT 2025).

# Chương 1

## Giới thiệu chung

Chương này xác định bài toán Hệ thống gợi ý hội thoại (CRS) và các thách thức còn tồn tại, tổng quan các hướng nghiên cứu liên quan, chỉ ra khoảng trống nghiên cứu, và trình bày mục tiêu cùng đóng góp kỹ thuật của khóa luận.

### 1.1 Đặt vấn đề và Động lực nghiên cứu

Khi khối lượng nội dung số bùng nổ và các nền tảng giải trí, thương mại điện tử ngày càng đa dạng, các hệ thống gợi ý truyền thống (Recommender Systems - RS) đang đối mặt với giới hạn cấu trúc khó vượt qua. Các mô hình lọc cộng tác và dựa trên nội dung vốn chỉ khai thác lịch sử tương tác một chiều (click, view, purchase), không thể nắm bắt sở thích tức thời hay các nhu cầu ngữ cảnh phức tạp phát sinh trong thực tế [4, 5]. Hệ thống gợi ý hội thoại (Conversational Recommender System - CRS) ra đời để lấp đầy khoảng trống này: thông qua giao tiếp bằng ngôn ngữ tự nhiên, CRS cho phép hệ thống chủ động làm rõ nhu cầu của người dùng và cung cấp các gợi ý có tính thuyết phục qua nhiều lượt tương tác.

Sự trưởng thành của các Mô hình ngôn ngữ lớn (Large Language Models - LLMs) mở ra cơ hội lớn cho CRS nhờ năng lực hiểu và sinh ngôn ngữ tự nhiên vượt trội. Tuy nhiên, ứng dụng trực tiếp LLM vào CRS bộc lộ hai rào cản cốt lõi. Về tính tin cậy, LLM sinh văn bản theo phân phối xác suất chứ không truy xuất dữ liệu thực tế, khiến chúng dễ phát sinh hiện tượng "ảo giác" (hallucination), tự bịa ra các thực thể không tồn tại như

tên phim, đạo diễn hay năm phát hành. Về hiệu quả vận hành, việc tinh chỉnh LLM cho mỗi thay đổi dữ liệu tốn kém tài nguyên; ngược lại, nếu đưa toàn bộ tri thức vào context window thì chi phí API và độ trễ tăng vọt, ảnh hưởng trực tiếp đến khả năng phản hồi thời gian thực.

Để giải quyết bài toán ảo giác mà không cần huấn luyện lại mô hình, các kiến trúc kết hợp RAG (Retrieval-Augmented Generation) với Đồ thị tri thức (Knowledge Graph - KG) đã được đề xuất. Tiêu biểu trong số này là kiến trúc GRACE (Graph-Reasoning Augmented Conversational Engine) [12], nghiên cứu tiền đề của chính tác giả đã được công bố tại SOICT 2025, vận hành bằng cách đối soát (grounding) kết quả sinh bởi LLM với dữ liệu thực trong KG thông qua cơ chế Truy xuất lai. Mặc dù giải quyết hiệu quả vấn đề ảo giác, GRACE và các hệ thống RAG tương tự vẫn hoạt động theo kiến trúc đường ống tĩnh (Static Pipeline), bộc lộ hạn chế rõ rệt khi đối mặt với các truy vấn yêu cầu suy luận đa bước hoặc chứa các ràng buộc mâu thuẫn, tình huống phổ biến trong hội thoại thực tế.

Điểm mấu chốt bị thiếu trong các hệ thống RAG tĩnh là cơ chế phản hồi nội bộ: khi một bước xử lý thất bại (truy xuất rỗng, thực thể sai), toàn bộ pipeline bị tắc mà không có khả năng tự điều chỉnh. Đây là động lực để khóa luận đề xuất hệ thống **ARGOS** (Adaptive Reasoning and Graph Orchestration System), một khung làm việc đa tác tử có khả năng tự phát hiện điểm nghẽn, điều chỉnh chiến lược truy xuất và duy trì mạch hội thoại ngay cả trong các tình huống phức tạp.

## 1.2 Tổng quan các nghiên cứu liên quan

Nghiên cứu về Hệ thống Gợi ý Hội thoại đã trải qua ba thế hệ tiếp cận rõ rệt, mỗi thế hệ giải quyết hạn chế của thế hệ trước nhưng đồng thời mang theo những ràng buộc mới.

### 1.2.1 Hệ thống CRS truyền thống tăng cường tri thức

Các hệ thống CRS thế hệ đầu mô hình hóa luồng hội thoại bằng mạng nơ-ron hồi quy (RNN) để trích xuất sở thích và thực hiện gợi ý. Mô hình nền tảng ReDial [4] kết hợp bộ tự mã hóa (autoencoder) với phân tích cảm xúc, song thường sinh ra các phản hồi "an toàn", chung chung và thiếu cá nhân hóa.

Nhận thấy hạn chế này, các nghiên cứu tiếp theo tích hợp Đồ thị Tri thức (KG) làm nguồn dữ liệu bổ trợ:

- **KBRD [6]:** Ánh xạ thực thể trong hội thoại vào đồ thị DBpedia và dùng GNN để lan truyền thông tin qua các quan hệ liên kết, cải thiện đáng kể độ chính xác so với ReDial.
- **KGSF [7]:** Giải quyết sự mất kết nối giữa không gian từ vựng và không gian thực thể bằng cơ chế hợp nhất thông tin tương hỗ (Mutual Information Maximization), đồng bộ hóa biểu diễn giữa KG và mạng đồ thị từ vựng.
- **CR-Walker [13] và KERL [8]:** Chuyển từ tính toán vector thuần túy sang mô hình hóa suy luận như chuỗi duyệt cây trên đồ thị. KERL còn sử dụng Học tăng cường (Reinforcement Learning) để tìm đường đi tối ưu trên KG.

Hạn chế cốt lõi của thể hệ này nằm ở module NLU cứng nhắc: phụ thuộc vào thuật toán nhận diện thực thể theo quy tắc khớp chuỗi (exact match), dễ bỏ sót thông tin khi người dùng diễn đạt ẩn ý hoặc dùng cách nói không chuẩn. Ngoài ra, mỗi tập dữ liệu mới đòi huấn luyện lại từ đầu với chi phí tài nguyên lớn.

### 1.2.2 Mô hình Ngôn ngữ Lớn trong Hệ thống gợi ý

LLM (GPT-4, LLaMA, Gemini) thay thế toàn bộ các module NLU và NLG truyền thống, cho phép hiểu ngữ cảnh phức tạp mà không cần quy tắc thủ công. Ba hướng tiếp cận chính được phân loại theo mức độ can thiệp vào trọng số mô hình:

- **Zero/Few-Shot Prompting:** LLM-ReDial [9] và LLM-ZCR [14] giữ nguyên trọng số (frozen weights), chỉ thiết kế prompt. Triển khai nhanh nhưng hoàn toàn phụ thuộc vào trí nhớ tham số của LLM, dẫn đến ảo giác nghiêm trọng khi domain knowledge không được cập nhật.
- **Fine-tuning:** TallRec [10] đóng gói bài toán thành phân loại nhị phân; ReFICR [11] dùng instruction tuning. Cả hai cải thiện độ chính xác nhưng kém linh hoạt khi có item mới, và chi phí tái huấn luyện liên tục là rào cản thực tiễn [15].

- **Knowledge-Augmented Prompting:** UniCRS [16], MESE [17] và PECRS [18] tiêm tri thức đồ thị vào context của LLM. Giải pháp trung dung nhưng bị giới hạn bởi context window, chỉ có thể trích xuất đồ thị ở độ sâu 1-hop, đánh mất khả năng suy luận đa bước.

### 1.2.3 Các kiến trúc lai và tăng cường truy xuất (RAG)

RAG biến LLM từ "người học thuộc lòng" thành "người tra cứu tài liệu theo thời gian thực", vừa giảm ảo giác vừa tránh được chi phí fine-tuning:

- **DCRS [19]:** Dùng bộ truy xuất tương phản (Contrastive Retriever) để ánh xạ tín hiệu cộng tác (item-item) vào không gian vector của LLM, song vẫn gặp khó khăn với các truy vấn ngôn ngữ tự do.
- **COLA [20]:** Cải thiện tích hợp cộng tác từ giai đoạn huấn luyện sớm, nhưng phạm vi chỉ xoay quanh ReDial.
- **G-CRS [21]:** Tiên phong dùng RAG trực tiếp trên KG: truy vấn ý định người dùng để quét KG, trích xuất mạng con (subgraph) liên quan rồi giao LLM đánh giá.

Điểm nghẽn chung của các kiến trúc RAG là khâu re-ranking: đưa hàng chục ứng viên kèm metadata vào Generative LLM làm tăng vọt độ trễ và chi phí API. Kiến trúc GRACE [12] đã giải quyết điểm này bằng cách kết hợp Hybrid Retrieval với cơ chế re-ranking dùng Generative LLM theo lô (batching), đạt hiệu năng vượt trội. Tuy nhiên, toàn bộ các hệ thống kể trên, kể cả GRACE, đều hoạt động theo đường ống tĩnh: dữ liệu chỉ chảy một chiều và không có cơ chế xử lý khi pipeline thất bại giữa chừng.

## 1.3 Khoảng trống nghiên cứu và Sự cần thiết của hệ thống đa tác tử

Bảng 1.1 hệ thống hóa các hướng tiếp cận CRS theo bốn chiều so sánh. Nhìn vào bức tranh tổng thể, một hạn chế cấu trúc xuyên suốt tất cả các thể hệ: **tất cả đều vận hành theo cơ chế đường ống tĩnh (Static Pipeline).**

Bảng 1.1: Tổng hợp các nghiên cứu liên quan trong lĩnh vực Hệ thống gợi ý hội thoại.

| Phân loại                             | Mô hình tiêu biểu                  | Phương pháp cốt lõi                                             | Dataset                  | Cần huấn luyện |
|---------------------------------------|------------------------------------|-----------------------------------------------------------------|--------------------------|----------------|
| <b>Truyền thống &amp; Tích hợp KG</b> | ReDial [4]                         | Bộ tự mã hóa RNN kết hợp phân tích cảm xúc.                     | ReDial                   | Có             |
|                                       | KBRD [6], KGSF [7]                 | Tích hợp KG làm giàu biểu diễn người dùng.                      | ReDial, IN-SPIRED        | Có             |
|                                       | CR-Walker [13], KERL [8]           | Suy luận trên cấu trúc cây của KG và văn bản.                   | ReDial, IN-SPIRED        | Có             |
| <b>LLM (Prompt/ Fine-tune)</b>        | LLM-ZCR [14], LLM-ReDial [9]       | Prompt zero-shot/few-shot giữ đúng bằng trọng số LLM.           | ReDial                   | Không          |
|                                       | TallRec [10], ReFICR [11]          | Tinh chỉnh toàn bộ trọng số (Fine-tune).                        | Nhiều                    | Có             |
|                                       | UniCRS [16], MESE [17], PECRS [18] | Tăng cường kiến thức qua trích xuất KG tĩnh.                    | ReDial, IN-SPIRED        | Linh hoạt      |
| <b>Kiến trúc Lai &amp; RAG</b>        | DCRS [19]                          | Bộ truy xuất tương phản (Contrastive Retriever).                | ReDial, IN-SPIRED        | Có             |
|                                       | COLA [20]                          | Tăng cường cộng tác huấn luyện từ đầu.                          | ReDial                   | Có             |
|                                       | G-CRS [21]                         | Graph RAG kết hợp với LLM nền tảng.                             | ReDial, IN-SPIRED        | Không          |
|                                       | <b>GRACE (Baseline)</b>            | <b>Truy xuất lai (Sem+Content+Graph) kết hợp LLM Re-ranker.</b> | <b>ReDial, IN-SPIRED</b> | <b>Không</b>   |

Trong các pipeline tĩnh, dữ liệu chảy một chiều: phân tích ý định → truy xuất KG → xếp hạng → sinh phản hồi. Khi bất kỳ bước nào gặp sự cố (truy xuất không trả kết quả

vì điều kiện quá khắt khe, hoặc LLM trích xuất nhầm thực thể), toàn bộ pipeline tắc mà không có cơ chế tự điều chỉnh. Kết quả là hệ thống hoặc trả về tập rỗng, hoặc sinh gợi ý chất lượng thấp, làm gián đoạn mạch hội thoại.

Phân tích kỹ hơn, hai khoảng trống nghiên cứu cụ thể được xác định:

**(1) Thiếu cơ chế tự kiểm định (Self-Correction):** Không có hệ thống CRS hiện tại nào có khả năng phát hiện và sửa lỗi nội bộ trong quá trình xử lý. Ví dụ cụ thể: nếu LLM trích xuất sai tên thực thể và gợi ý không tồn tại trong KG, hệ thống không thể nhận biết và loại bỏ ứng viên đó trước khi trả về người dùng.

**(2) Thiếu khả năng thỏa hiệp ràng buộc (Constraint Relaxation):** Khi người dùng đặt ra các điều kiện mâu thuẫn hoặc quá hạn hẹp, chúng không thể tự quyết định nên nới lỏng ràng buộc nào để đưa ra gợi ý xấp xỉ; thay vào đó chúng trả về kết quả rỗng, một trải nghiệm người dùng rất kém.

Trong khi đó, nghiên cứu về kiến trúc đa tác tử dựa trên LLM [22] đã chứng minh rằng việc phân tách hệ thống thành các tác tử chuyên biệt với vòng lặp phản hồi (feedback loop) có thể xử lý hiệu quả các bài toán đòi hỏi suy luận đa bước và tự sửa lỗi. Tuy nhiên, hướng tiếp cận này chưa được áp dụng vào bài toán CRS kết hợp Đồ thị tri thức, đây chính là khoảng trống mà khóa luận đề xuất lấp đầy thông qua hệ thống **ARGOS**.

## 1.4 Mục tiêu nghiên cứu

Khóa luận đặt ra bốn mục tiêu nghiên cứu:

- **Mục tiêu 1:** Phân tích và định lượng các điểm nghẽn của kiến trúc GRACE trong bài toán CRS để tạo cơ sở thực nghiệm cho việc đề xuất cơ chế mới.
- **Mục tiêu 2:** Thiết kế kiến trúc đa tác tử ARGOS với cơ chế phân phối nhiệm vụ và vòng lặp phản hồi nội bộ, giải quyết các hạn chế đó mà không yêu cầu huấn luyện lại mô hình.
- **Mục tiêu 3:** Phát triển và tích hợp cơ chế Critic-Relaxation (bao gồm Critic Agent kiểm định tính xác thực và Relaxation Agent nới lỏng ràng buộc) vào vòng lặp suy luận của ARGOS.



- **Mục tiêu 4:** Đánh giá toàn diện ARGOS trên hai bộ dữ liệu chuẩn ReDial và INSPIRED, so sánh với các mô hình tiên tiến nhất hiện nay, và phân tích định lượng đóng góp của từng thành phần.

## 1.5 Đóng góp của khóa luận

Khóa luận mang lại ba đóng góp kỹ thuật mới cho lĩnh vực CRS:

1. **Kiến trúc đa tác tử ARGOS:** Lần đầu tiên áp dụng mô hình Multi-Agent với vòng lặp suy luận-hành động vào hệ thống CRS kết hợp Đồ thị tri thức. ARGOS phân tách pipeline thành năm tác tử chuyên biệt (Profiler Agent, Orchestrator, Graph Agent, Critic Agent, Relaxation Agent) và sử dụng Dynamic Weighting để điều phối chiến lược truy xuất theo hình thái ngôn ngữ của truy vấn, cơ chế chưa xuất hiện trong các hệ thống CRS prior work.
2. **Cơ chế Critic-Relaxation:** Phương pháp tự kiểm định và thỏa hiệp ràng buộc tự động trong CRS. Critic Agent đối chiếu trực tiếp ứng viên với Đồ thị tri thức để loại bỏ hallucination; Relaxation Agent áp dụng thuật toán phân cấp ưu tiên (Sacrifice Hierarchy) để tái cấu trúc truy vấn khi gặp tập kết quả rỗng. Đây là đóng góp kỹ thuật độc lập, có thể tách rời và ứng dụng cho các hệ thống RAG khác ngoài CRS.
3. **Kết quả thực nghiệm và mã nguồn mở:** ARGOS cải thiện đáng kể so với kiến trúc tiền đề GRACE và đạt kết quả cạnh tranh với các hệ thống tiên tiến trên ReDial và INSPIRED về chỉ số Recall@K. Toàn bộ mã nguồn và dữ liệu thực nghiệm được công bố công khai tại <https://github.com/DatPhan06/GRACE>, tạo nền tảng cho các nghiên cứu tiếp theo trong lĩnh vực.

## Kết luận chương

Chương 1 đã xác lập nền tảng định hướng cho toàn bộ khóa luận. Xuất phát từ hạn chế cốt lõi của các hệ thống CRS hiện tại (thiếu tính linh hoạt của thể hệ truyền thống và hiện tượng ảo giác của các LLM thuần túy), phân tích cho thấy các kiến trúc RAG, dù đã cải thiện đáng kể, vẫn bị ràng buộc bởi cơ chế đường ống tĩnh. Khoảng trống về khả

năng tự kiểm định và thỏa hiệp ràng buộc là cơ sở kỹ thuật rõ ràng để đề xuất hệ thống ARGOS. Các kiến thức nền tảng cần thiết để hiểu kiến trúc ARGOS sẽ được trình bày chi tiết trong Chương 2.

## Chương 2

# Kiến thức nền tảng

Chương này trình bày các khái niệm và kỹ thuật nền tảng mà hệ thống ARGOS được xây dựng trên đó. Mỗi mục được trình bày ở mức độ đủ để người đọc theo dõi các phân tích kiến trúc ở các chương sau: từ định nghĩa bài toán CRS, cấu trúc Đồ thị tri thức, kỹ thuật LLM và RAG, đến mô hình Đa tác tử và các thang đo đánh giá.

### 2.1 Định nghĩa bài toán Hệ thống Gợi ý Hội thoại (CRS)

Không giống các hệ thống gợi ý truyền thống vốn chỉ nhận đầu vào là lịch sử tương tác ngầm (implicit feedback), CRS tương tác trực tiếp với người dùng qua ngôn ngữ tự nhiên để khai thác sở thích một cách chủ động [4, 5]. Trọng tâm là khả năng duy trì mạch hội thoại đa lượt: hệ thống đặt câu hỏi để làm rõ nhu cầu (preference elicitation), cập nhật hiểu biết về người dùng sau mỗi lượt và đưa ra gợi ý kèm lý giải thuyết phục. CRS đặc biệt có giá trị trong tình huống cold-start, khi hệ thống chưa có dữ liệu tương tác lịch sử của người dùng mới.

Về kiến trúc, một CRS truyền thống gồm ba module cốt lõi: NLU (Natural Language Understanding) trích xuất ý định, DST (Dialogue State Tracking) theo dõi trạng thái hội thoại, và NLG (Natural Language Generation) sinh phản hồi.

Bài toán CRS được hình thức hóa như sau: cho tập item  $\mathcal{I}$  và ngữ cảnh hội thoại tại lượt  $t$ ,  $C_t = \{u_1, s_1, \dots, u_t\}$ , trong đó  $u_i$  là phát ngôn người dùng và  $s_i$  là phản hồi hệ thống ở lượt  $i$ , hệ thống thực hiện hai nhiệm vụ song song:

1. **Gợi ý (Recommendation):** Dự đoán tập con  $I_{rec} \subset \mathcal{I}$  phù hợp nhất với hồ sơ sở thích động của người dùng, liên tục cập nhật theo phản hồi chấp nhận/từ chối ở các lượt trước.
2. **Sinh hội thoại (Dialogue Generation):** Sinh phản hồi  $s_t$ : hoặc là câu hỏi khai thác thêm thông tin khi độ tin cậy chưa đủ, hoặc là lời giới thiệu tích hợp các item trong  $I_{rec}$  cùng lý do giải thích để tăng mức độ tin tưởng của người dùng.

Hình 2.1 minh họa một đoạn hội thoại CRS điển hình, cho thấy hệ thống luân phiên đặt câu hỏi khai thác sở thích và đưa ra gợi ý qua nhiều lượt trao đổi với người dùng.

## 2.2 Đồ thị Tri thức (Knowledge Graph - KG)

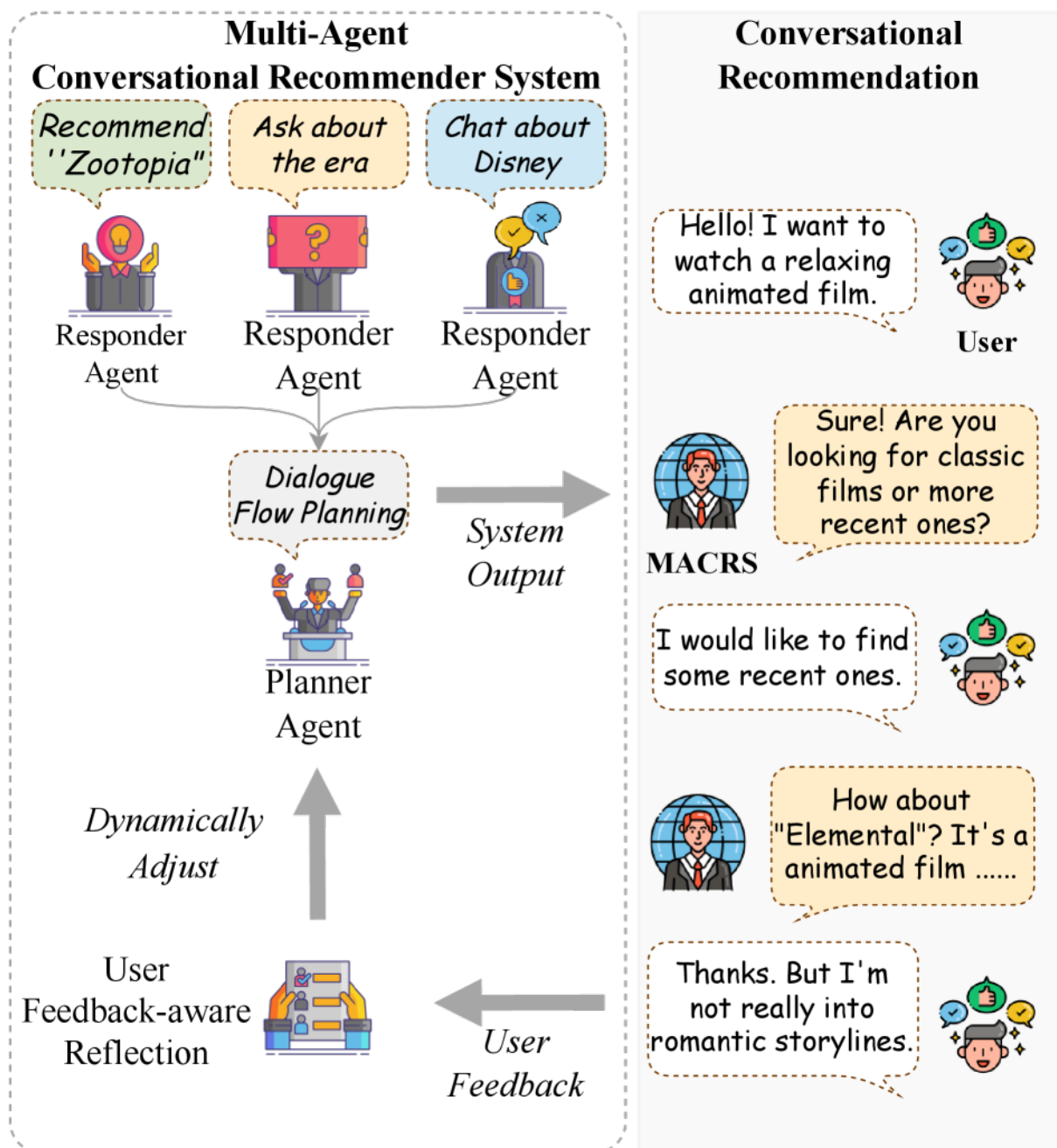
Đồ thị tri thức (KG) biểu diễn tri thức thực thể dưới dạng tập hợp các bộ ba có hướng:

$$\mathcal{G} = \{(h, r, t) \mid h, t \in \mathcal{E}, r \in \mathcal{R}\} \quad (2.1)$$

trong đó  $\mathcal{E}$  là tập thực thể (Phim, Đạo diễn, Thể loại, Diễn viên),  $\mathcal{R}$  là tập quan hệ (đạo\_diễn\_bởi, thuộc\_thể\_loại, đóng\_vai),  $h$  (head) và  $t$  (tail) là thực thể gốc và đích,  $r$  là quan hệ có hướng nối giữa chúng [7].

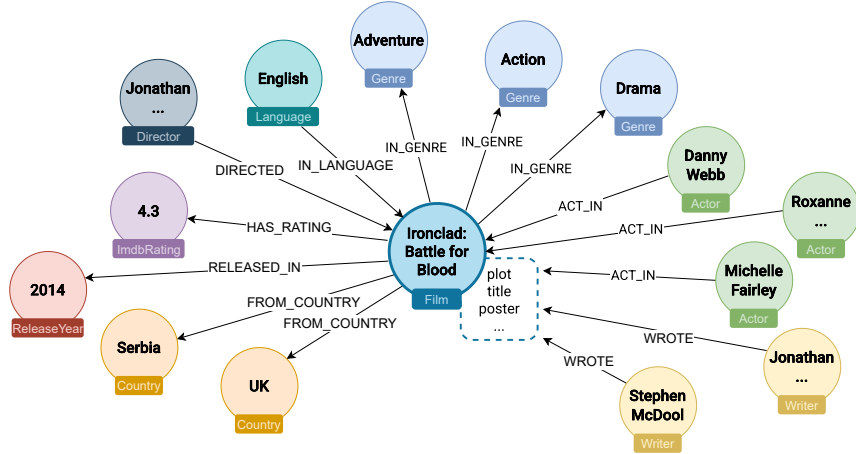
Trong ngữ cảnh CRS, dữ liệu tương tác người dùng - item thường rất thưa thớt (data sparsity). KG đóng ba vai trò bổ sung mà lịch sử tương tác đơn thuần không thể cung cấp:

- **Nguồn dữ liệu thực chứng (Grounding):** Mọi thực thể được gợi ý đều có thể kiểm tra sự tồn tại trực tiếp trong KG, là nền tảng để loại bỏ hallucination từ LLM.
- **Suy luận đa bước (Multi-hop Reasoning):** Duyệt qua các cạnh đồ thị ở độ sâu  $> 1$  cho phép phát hiện sở thích tiềm ẩn mà lịch sử tương tác trực tiếp không thể hiện (ví dụ: Phim  $\rightarrow$  Diễn viên  $\rightarrow$  Phim khác cùng diễn viên).
- **Nền tảng cho Truy xuất lai (Hybrid Retrieval):** KG hiện đại lưu trữ song song cả liên kết cấu trúc và vector nhúng (embeddings) của nội dung văn bản (tóm tắt cốt truyện, mô tả thực thể), cho phép kết hợp truy xuất ngữ nghĩa và truy xuất đồ thị trong cùng một hệ thống.



Hình 2.1: Ví dụ hội thoại nhiều lượt trong Hệ thống gợi ý hội thoại. Hệ thống chủ động đặt câu hỏi làm rõ sở thích, cập nhật trạng thái hội thoại và đưa ra gợi ý kèm giải thích qua từng lượt tương tác [1].

Hình 2.2 minh họa cụ thể cách một node Phim được tổ chức trong KG, với các cạnh có hướng nối tới các thực thể liên quan như Đạo diễn, Diễn viên và Thể loại, thể hiện trực quan cấu trúc bộ ba  $(h, r, t)$  được định nghĩa ở trên.



Hình 2.2: Ví dụ cấu trúc một node Phim trong Đồ thị tri thức, liên kết với các thực thể Đạo diễn, Diễn viên, Thể loại và các thuộc tính định lượng. Mỗi bộ ba  $(h, r, t)$  biểu diễn một quan hệ có hướng giữa hai thực thể trong đồ thị.

## 2.3 Mô hình Ngôn ngữ Lớn (LLMs) và Kỹ thuật RAG

### 2.3.1 Mô hình Ngôn ngữ Lớn (LLMs)

LLM là các mô hình Transformer được huấn luyện trên khối lượng văn bản khổng lồ, có khả năng hiểu và sinh ngôn ngữ tự nhiên theo ngữ cảnh [23]. Trong hệ thống CRS, LLM đảm nhận ba chức năng chính: (1) phân tích ý định và trích xuất ràng buộc từ lịch sử hội thoại, (2) suy luận về độ phù hợp của các ứng viên, và (3) sinh phản hồi ngôn ngữ tự nhiên [24].

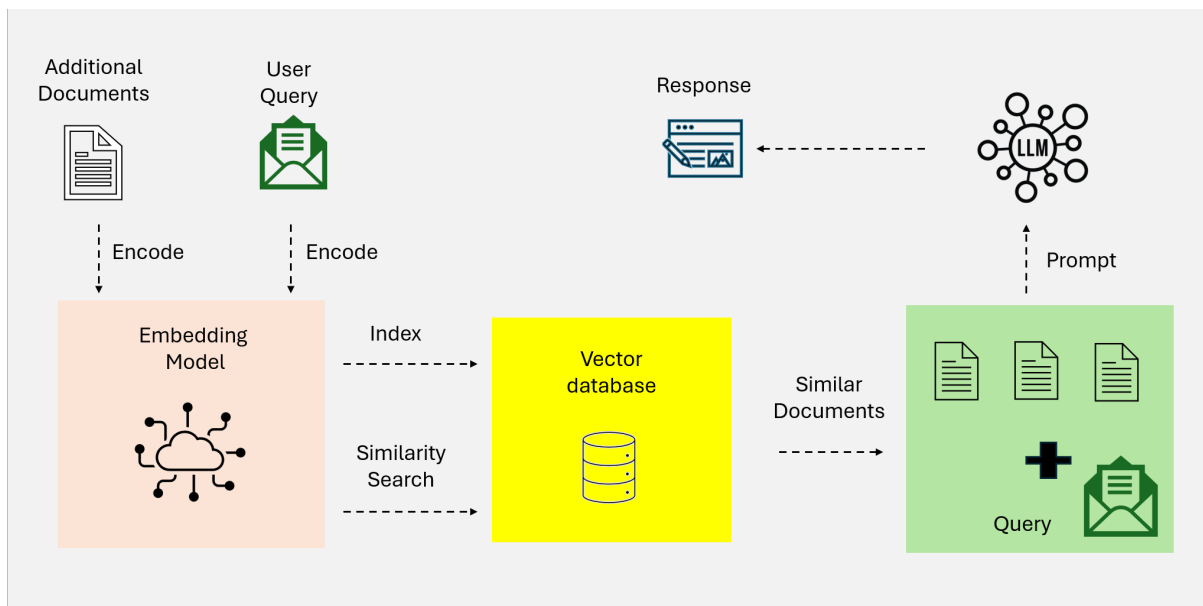
Hạn chế căn bản của LLM trong CRS là *hallucination*: vì sinh văn bản theo phân phối xác suất, LLM có thể tự tạo ra các thực thể không tồn tại (phim bịa, đạo diễn sai) nhưng hoàn toàn hợp lý về mặt ngôn ngữ. Đây là lý do cần neo giữ (ground) đầu ra của LLM vào một nguồn dữ liệu thực chứng bên ngoài, vai trò mà KG đảm nhận trong hệ thống ARGOS.

### 2.3.2 Kỹ thuật RAG (Retrieval-Augmented Generation)

RAG giải quyết hạn chế của LLM bằng cách tách biệt lưu trữ tri thức (trong cơ sở dữ liệu ngoài) khỏi năng lực suy luận (của LLM) [2]. Thay vì học thuộc dữ liệu vào trọng số, LLM nhận tri thức liên quan theo thời gian thực qua hai pha:

1. **Truy xuất (Retrieval):** Dựa trên truy vấn  $q$ , tìm kiếm tập tài liệu/thực thể  $D$  liên quan từ kho dữ liệu ngoài (KG, vector database).
2. **Sinh có điều kiện (Conditioned Generation):** LLM sinh phản hồi dựa trên  $q$  và  $D$  kết hợp, câu trả lời được neo giữ vào dữ liệu thực chứng thay vì chỉ dựa vào phân phối tham số.

Luồng hoạt động tổng thể của pipeline RAG được minh họa trong Hình 2.3.



Hình 2.3: Pipeline RAG cơ bản gồm ba giai đoạn: *Retrieval* (truy xuất tài liệu liên quan từ kho dữ liệu ngoài), *Augmentation* (tích hợp tài liệu vào ngữ cảnh) và *Generation* (LLM sinh câu trả lời được neo giữ vào dữ liệu thực chứng) [2].

RAG đặc biệt phù hợp với bài toán CRS vì ba lý do thực tiễn: (i) danh mục item thay đổi liên tục: RAG chỉ cần cập nhật KG mà không cần fine-tune lại LLM; (ii) mỗi gợi ý đều cần có cơ sở thực chứng: đường dẫn truy xuất của RAG cung cấp trực tiếp khả năng giải thích (explainability); (iii) chi phí thấp hơn đáng kể so với fine-tuning khi quy mô dữ liệu lớn.

## 2.4 Kiến trúc Hệ thống Đa tác tử (Multi-Agent System) và ReAct

### 2.4.1 Hệ thống Đa tác tử dựa trên LLM

Kiến trúc đa tác tử dựa trên LLM phân tách một hệ thống phức tạp thành nhiều tác tử (agents) tự trị, mỗi tác tử chuyên trách một nhiệm vụ cụ thể với prompt riêng và quyền truy cập vào các công cụ được chỉ định [22]. Nghiên cứu gần đây cho thấy cách tiếp cận này vượt trội so với kiến trúc đơn tác tử trong các bài toán đòi hỏi phân tích đa chiều, tự sửa lỗi và phối hợp giữa các nguồn thông tin dị dạng, đặc điểm phù hợp với tính chất phức tạp của hội thoại gợi ý thực tế.

### 2.4.2 Vòng lặp ReAct (Reasoning and Acting)

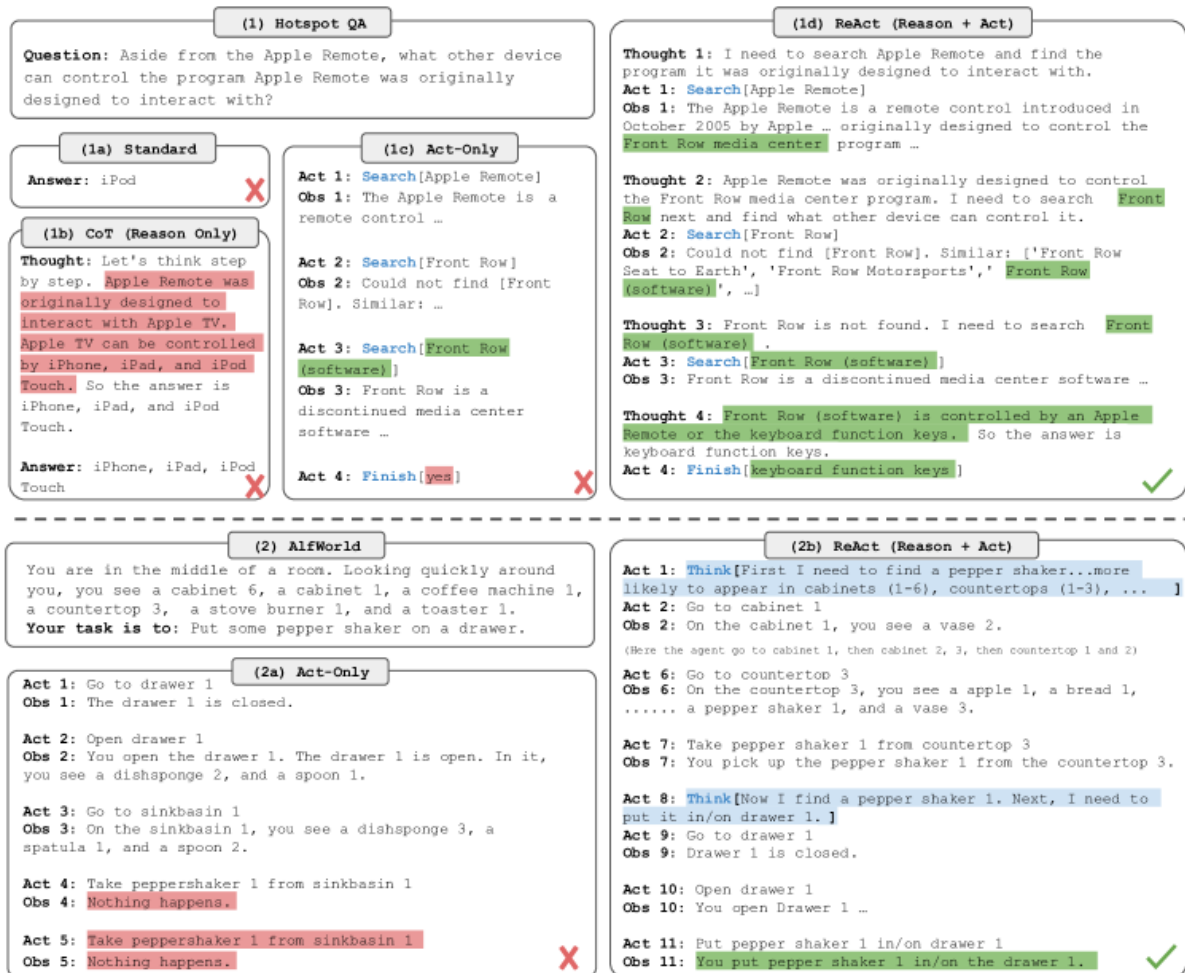
ReAct [3] là framework cho phép tác tử xen kẽ giữa suy luận (Thought) và hành động tương tác với môi trường (Action-Observation) trong một vòng lặp liên tục:

- **Thought:** Tác tử phân tích trạng thái hiện tại và lập kế hoạch bước tiếp theo.
- **Action:** Gọi công cụ cụ thể (truy vấn KG, gọi API) với tham số phù hợp.
- **Observation:** Nhận kết quả từ công cụ, cập nhật ngữ cảnh và lặp lại.

Hình 2.4 so sánh trực quan vòng lặp ReAct với các phương pháp prompting khác, làm rõ cách xen kẽ giữa Thought và Action-Observation tạo nên khả năng tự điều chỉnh mà các phương pháp đơn thuần không có.

ReAct phù hợp đặc biệt với bài toán CRS vì bản chất tương tác đa lượt của hội thoại đòi hỏi hệ thống phải liên tục quan sát, điều chỉnh, hành động trong vòng lặp, chứ không thể xử lý một lần rồi kết thúc như pipeline tĩnh. Cụ thể, khi một bước truy xuất KG thất bại, ReAct cho phép tác tử nhận biết sự cố qua Observation, suy luận nguyên nhân qua Thought tiếp theo, và thử chiến lược truy xuất khác qua Action mới, thay vì dừng hẳn và trả về kết quả rỗng. Đây chính xác là cơ chế mà các pipeline tĩnh đang thiếu.





Hình 2.4: So sánh các phương pháp prompting LLM: Standard, Chain-of-Thought, Act-only và **ReAct**. ReAct xen kẽ giữa suy luận (Thought) và hành động tương tác với môi trường (Action-Observation), cho phép tác tử tự điều chỉnh chiến lược dựa trên phản hồi từ môi trường [3].

## 2.5 Bộ dữ liệu thực nghiệm

Toàn bộ quá trình đánh giá trong khóa luận sử dụng hai bộ dữ liệu hội thoại chuẩn của lĩnh vực CRS:

**ReDial** [4] là bộ dữ liệu hội thoại gợi ý phim tiếng Anh được thu thập qua Amazon Mechanical Turk, gồm hơn 10.000 hội thoại đa lượt giữa người hỏi và người trả lời. Mỗi hội thoại chứa ít nhất một lượt gợi ý phim với nhãn xác nhận (accepted/rejected), tạo thành ground truth rõ ràng cho đánh giá Recall@K.

**INSPIRED** [25] là bộ dữ liệu hội thoại gợi ý phim tập trung vào chiến lược thuyết phục xã hội (social influence strategies), gồm 1.001 hội thoại với ngôn ngữ tự nhiên, cảm xúc và phong cách giao tiếp đa dạng hơn ReDial. Bộ dữ liệu này kiểm tra khả năng hiểu ngôn ngữ linh hoạt và hội thoại phi cấu trúc của hệ thống.

Hai bộ dữ liệu bổ sung cho nhau: ReDial đánh giá hiệu năng ở quy mô lớn với phân phối cân bằng, còn INSPIRED kiểm tra khả năng xử lý các truy vấn cảm xúc và ngôn ngữ đời thường. Giao thức đánh giá chi tiết (chiến lược lấy mẫu, kiểm định thống kê) được trình bày trong Chương 5.

## 2.6 Các phương pháp đánh giá hệ thống (Evaluation Metrics)

Nghiên cứu này sử dụng thang đo tiêu chuẩn **Recall@K** trong lĩnh vực gợi ý để đánh giá module Recommendation của ARGOS. Thang đo này phù hợp với bài toán CRS vì mỗi lượt hội thoại thường chỉ có một mục ground truth duy nhất, nên việc đo tỷ lệ xuất hiện trong top-K là chỉ số đánh giá trực tiếp và có ý nghĩa nhất.

**Recall@K** đo tỷ lệ item mục tiêu (ground truth) xuất hiện trong top-K gợi ý. Ký hiệu  $V_u$  là tập item thực tế người dùng quan tâm,  $R_u@K$  là top-K gợi ý của hệ thống:

$$\text{Recall@K} = \frac{1}{|U|} \sum_{u \in U} \frac{|R_u@K \cap V_u|}{|V_u|} \quad (2.2)$$

Thang đo được tính tại bốn ngưỡng  $K \in \{1, 5, 10, 50\}$  trên hai bộ dữ liệu chuẩn

ReDial [4] và INSPIRED [25], nhất quán với giao thức đánh giá của các hệ thống SOTA được đối sánh trong Chương 5.

## Kết luận chương

Chương 2 đã thiết lập bốn nhóm kiến thức cốt lõi phục vụ cho việc hiểu và đánh giá hệ thống ARGOS. Bài toán CRS xác lập không gian vấn đề với hai nhiệm vụ song song mà ARGOS phải giải quyết. Đồ thị tri thức là nguồn dữ liệu thực chứng trung tâm, đảm bảo mọi gợi ý đều có cơ sở kiểm chứng và là nền tảng cho Hybrid Retrieval. LLM kết hợp RAG cung cấp năng lực suy luận ngôn ngữ mà không cần fine-tuning khi dữ liệu thay đổi. Kiến trúc đa tác tử với ReAct là mô hình tổ chức cho phép thiết lập vòng lặp phản hồi nội bộ, thành phần then chốt mà các hệ thống RAG tĩnh đang thiếu, và là nền tảng kỹ thuật của ARGOS.

Chương 3 tiếp theo phân tích chi tiết mô hình tiền đề GRACE, nơi RAG kết hợp KG lần đầu được áp dụng vào CRS của chính tác giả, để định lượng chính xác các điểm nghẽn mà ARGOS cần khắc phục.

## Chương 3

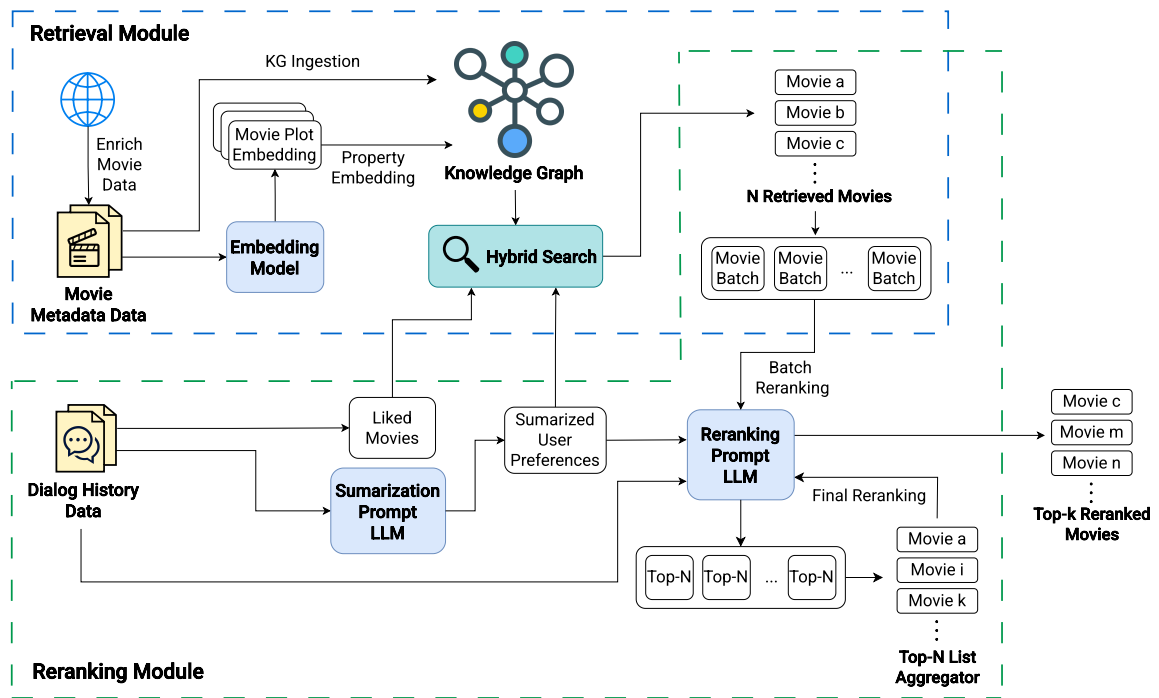
# Nghiên cứu tiền đề: Hệ thống GRACE

Chương này phân tích chi tiết hệ thống *GRACE* (Graph-Reasoning Augmented Conversational Engine), nghiên cứu tiền đề của chính tác giả đã được công bố tại hội nghị quốc tế SOICT 2025 [12]. Thay vì chỉ tóm tắt lại nội dung bài báo, chương tập trung làm rõ từng thành phần kỹ thuật cụ thể và chỉ ra các giới hạn cấu trúc mà *GRACE* chưa giải quyết được, đây là cơ sở trực tiếp để đề xuất hệ thống *ARGOS* trong chương tiếp theo.

### 3.1 Tổng quan kiến trúc GRACE

*GRACE* được thiết kế theo mô hình hai giai đoạn công nghiệp tiêu chuẩn: Tạo ứng viên (Candidate Generation) và Xếp hạng lại (Re-ranking). Toàn bộ quy trình hoạt động theo cơ chế đường ống tĩnh (Static Pipeline) và không yêu cầu tinh chỉnh (fine-tune) trọng số của bất kỳ mô hình ngôn ngữ nào.

Như minh họa trong Hình 3.1, luồng hoạt động của *GRACE* gồm bốn bước tuần tự: (1) LLM tóm tắt lịch sử hội thoại thành một hồ sơ thích (Preference Profile); (2) ba chiến lược truy xuất song song (ngữ nghĩa, nội dung, đồ thị) tạo ra tập ứng viên; (3) các tập được hợp nhất và xếp hạng bằng Weighted Reciprocal Rank Fusion; (4) Generative LLM đánh giá lại và chọn ra kết quả cuối cùng. Kiến trúc này đảm bảo mọi thực thể trong tập ứng viên đều tồn tại trong KG vì chúng được truy xuất trực tiếp, do đó loại trừ



Hình 3.1: Tổng quan kiến trúc hệ thống tiền đề GRACE, bao gồm hai module chính: LLM Retriever và LLM Re-ranker.

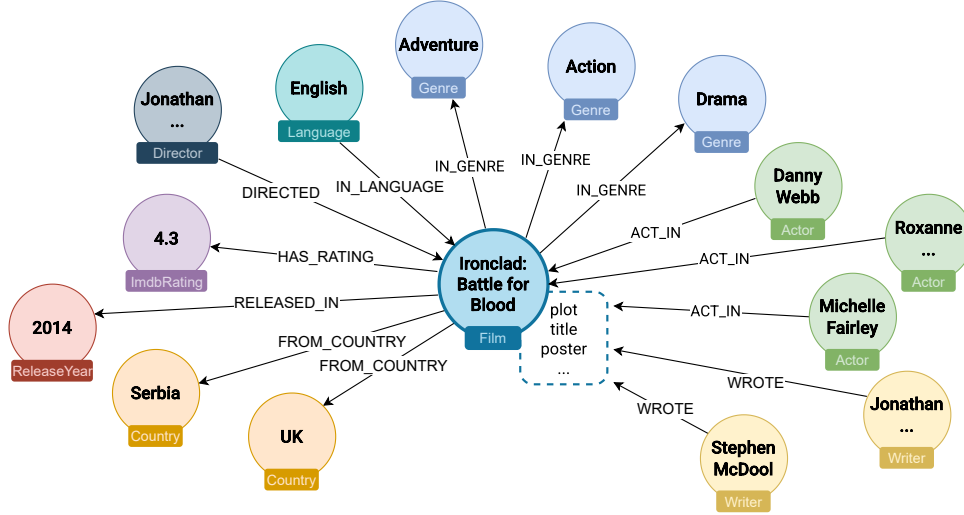
khả năng LLM tự bịa ra phim không tồn tại. Tuy nhiên, pipeline không kiểm tra liệu các ứng viên này có thỏa mãn toàn bộ ràng buộc cứng mà người dùng đặt ra trong hội thoại (danh sách loại trừ, ngưỡng điểm đánh giá) hay không; đây là khoảng trống mà Critic Agent trong ARGOS được thiết kế để lấp đầy.

## 3.2 Xây dựng Đồ thị tri thức (Knowledge Graph Construction)

Hiệu quả của GRACE phụ thuộc trực tiếp vào cấu trúc của Đồ thị tri thức (KG). Đồ thị này thống nhất siêu dữ liệu có cấu trúc và biểu diễn ngữ nghĩa cốt truyện vào cùng một không gian truy vấn.

Quy trình xây dựng KG gồm ba bước:

- **Tiền xử lý và Chuẩn hóa:** Chuyển đổi dữ liệu thô (năm phát hành, điểm số, thể loại, v.v.) sang định dạng sơ đồ thống nhất, xử lý các trường hợp nhập nhằng và trùng lặp thực thể.



Hình 3.2: Cấu trúc một node Phim (Film) trong Đồ thị tri thức. Node này liên kết trực tiếp với các thuộc tính (Năm, Điểm số, Thể loại) và các thực thể sáng tạo (Đạo diễn, Diễn viên).

- **Khởi tạo Đồ thị (Graph Construction):** Ánh xạ các thực thể phim và quan hệ vào cơ sở dữ liệu đồ thị Neo4j. Mỗi node Phim (*Film*) được kết nối với các node phụ trợ như Thể loại (*Genre*), Diễn viên (*Actor*), Đạo diễn (*Director*) (xem Hình 3.2).
- **Nhúng vector cốt truyện (Plot Embedding):** Gắn kèm vector nhúng dày đặc (dense embeddings) của đoạn tóm tắt cốt truyện trực tiếp vào node Phim tương ứng, cho phép truy vấn độ tương đồng cosine ngay bên trong đồ thị.

### 3.3 Tóm tắt hội thoại và xây dựng Hồ sơ sở thích (Dialogue Summarization)

Ký hiệu hội thoại đa lượt là  $C = \{u_1, u_2, \dots, u_T\}$  với  $u_t$  là phát ngôn thứ  $t$ . Mục tiêu của bước này là chưng cất  $C$  thành một chuỗi văn bản tự nhiên duy nhất  $\hat{P}$ , gọi là Hồ sơ sở thích (Preference Profile), tóm tắt toàn bộ nhu cầu người tìm kiếm.

Thay vì dùng phương pháp điền khung (slot-filling) cứng nhắc với các trường có cấu trúc như *likes*, *dislikes*, *constraints*, GRACE sử dụng một mẫu lời nhắc cố định UserPreference để buộc LLM sinh ra một đoạn văn xuôi mô tả duy nhất. Cách tiếp cận này tạo ra bản tóm tắt vừa đủ ngắn gọn, vừa dễ đọc, vừa thích hợp cho cả tìm kiếm từ

khóa lần neo giữ vào KG, đồng thời giữ được tính chịu lỗi cao trước ngôn ngữ đời thường và hội thoại nhiều.

Ví dụ từ tập dữ liệu *ReDial*: cho đoạn hội thoại  $C = [\text{Respondent: "Hi, what kind of movies do you like?"}, \text{Initiator: "I'm looking for a recommendation, I enjoyed A Nightmare on Elm Street (1984) when I was younger."}, \text{Respondent: "Oh, you like scary movies?"}]$ , LLM theo mẫu UserPreference sinh ra:  $\hat{P} = \text{"The seeker enjoys scary movies, particularly horror classics, and explicitly mentions A Nightmare on Elm Street (1984) as a movie they liked in the past. Their interest suggests a preference for suspenseful, frightening films, which can guide future recommendations."}$  Kết quả này cho thấy LLM không chỉ ghi nhận thực thể tường minh (tên phim) mà còn suy luận ra đặc trưng thể loại tiềm ẩn (genre affinity) từ ngữ cảnh.

Hồ sơ  $\hat{P}$  sau đó được dùng làm đầu vào trực tiếp cho cả ba luồng truy xuất trong §3.4.

### 3.4 Module Truy xuất Lai (Hybrid Retriever)

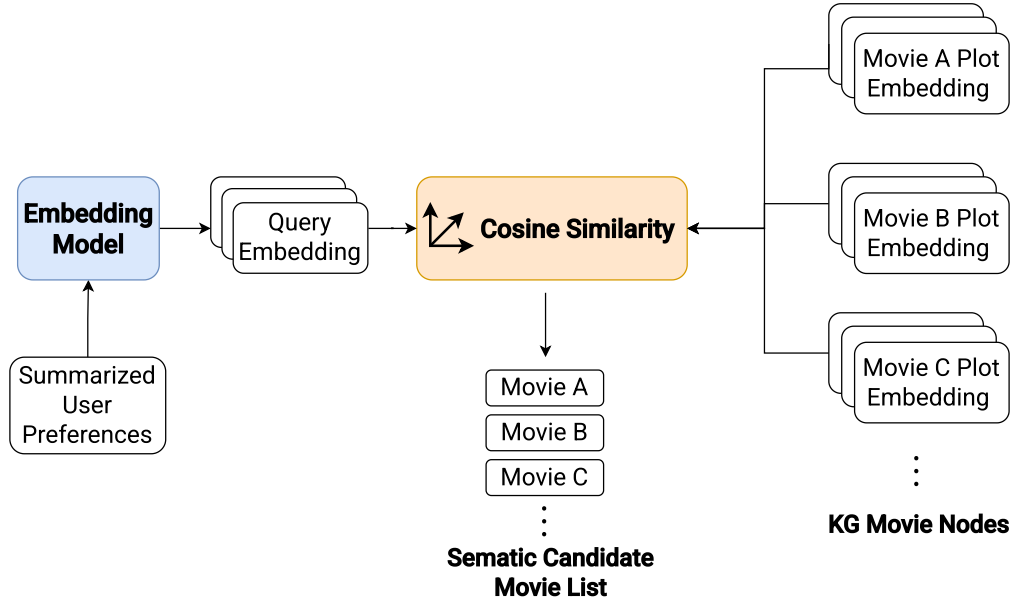
Cơ chế truy xuất của GRACE kết hợp ba luồng độc lập để đảm bảo độ bao phủ (Recall) tối đa cho tập ứng viên, trong đó mỗi luồng bổ trợ cho nhau về loại thông tin khai thác được.

#### 3.4.1 Truy xuất ngữ nghĩa (Semantic Search)

Hồ sơ sở thích được mã hóa thành vector truy vấn dày đặc  $\mathbf{q}$ , sau đó so sánh với vector cốt truyện  $\mathbf{e}_f$  tại mỗi node phim trong KG qua độ tương đồng Cosine:

$$s_f = \cos(\mathbf{q}, \mathbf{e}_f) = \frac{\mathbf{q} \cdot \mathbf{e}_f}{\|\mathbf{q}\|_2 \|\mathbf{e}_f\|_2} \quad (3.1)$$

trong đó  $\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2}$  là chuẩn Euclidean ( $\ell_2$ ). Top- $m$  phim có  $s_f$  cao nhất tạo thành tập kết quả  $M_{\text{sem}}$ . Luồng này đặc biệt hiệu quả khi nhu cầu người dùng mang tính chất thái, chủ đề hoặc cảm xúc khó biểu diễn bằng từ khóa cứng (Hình 3.3).



Hình 3.3: Quy trình truy xuất ngữ nghĩa dựa trên phép tính tương đồng Cosine giữa vector của câu truy vấn và vector cốt truyện phim.

### 3.4.2 Lọc theo nội dung (Content-based Filtering)

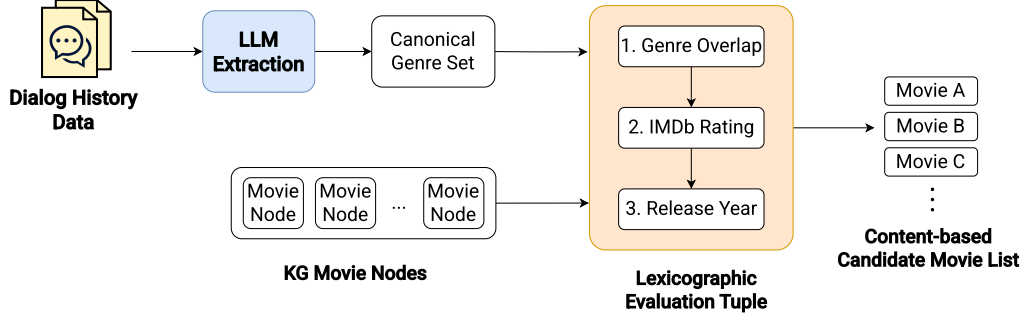
Để thực thi các ràng buộc tường minh, ngữ cảnh hội thoại được chuyển đổi thành tập thể loại chuẩn hóa  $G$  thông qua một bộ trích xuất LLM có ràng buộc (constrained LLM extractor), áp dụng ánh xạ từ đồng nghĩa (ví dụ: "*science fiction*"  $\mapsto$  "*sci-fi*"), chuẩn hóa không phân biệt chữ hoa/thường, và chấp nhận nhãn ghép (compound labels). Khi  $G$  không thể trích xuất đáng tin cậy, một bản đồ dự phòng keyword  $\rightarrow$  genre được dùng để đảm bảo độ phủ.

Mỗi phim  $f$  với tập thể loại  $\Gamma_f$  được đánh giá theo bộ giá trị xếp hạng từ điển (lexicographic tuple):

$$g_f = (|\Gamma_f \cap G|, r_f, y_f) \quad (3.2)$$

trong đó  $|\Gamma_f \cap G|$  là số thể loại trùng khớp (ưu tiên tuyệt đối),  $r_f$  là điểm IMDb, và  $y_f$  là năm phát hành. Cách sắp xếp theo  $y_f$  giảm dần mặc định ưu tiên phim mới hơn, một heuristic hợp lý trong phần lớn trường hợp nhưng có thể không phù hợp khi người dùng tìm phim cổ điển. Top- $m$  phim tạo thành tập  $M_{\text{con}}$  (Hình 3.4).





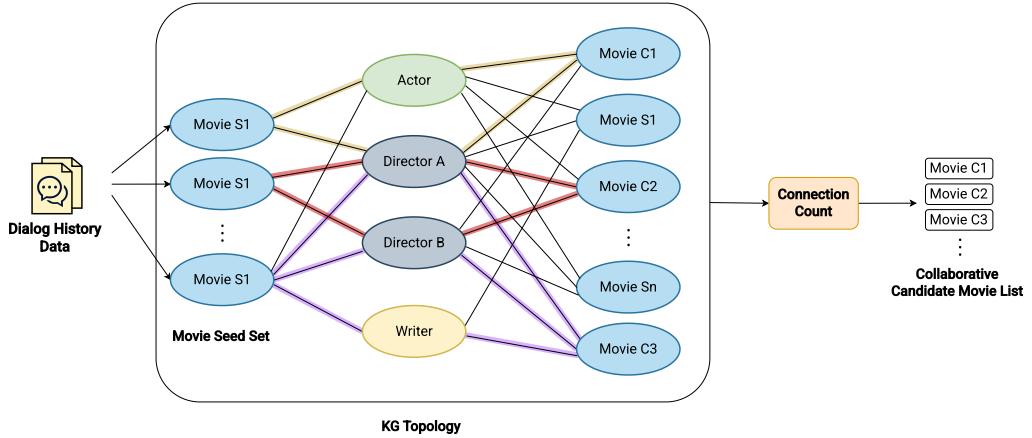
Hình 3.4: Cơ chế lọc theo nội dung dựa trên các thuộc tính cứng (Thể loại, Năm sản xuất, Điểm số).

### 3.4.3 Mở rộng đồ thị (Graph Expansion - Collaborative)

Xuất phát từ tập hạt giống (seed set)  $L$  gồm các phim người dùng đã đề cập tích cực, thuật toán duyệt đồ thị dọc theo các cạnh liên kết với Diễn viên, Đạo diễn hoặc Biên kịch để tìm phim lân cận. Điểm số cộng tác cho phim ứng viên  $f$  được tính:

$$S_{\text{col}}(f) = \sum_{s \in L} c(s, f) \quad (3.3)$$

trong đó  $c(s, f)$  là số thực thể sáng tạo chia sẻ chung giữa phim hạt giống  $s$  và phim ứng viên  $f$ . Các phim được sắp xếp theo tuple  $(S_{\text{col}}(f), r_f)$  và top- $m$  tạo thành tập  $M_{\text{col}}$  (Hình 3.5).



Hình 3.5: Thuật toán mở rộng đồ thị dựa trên việc chia sẻ các node đội ngũ sáng tạo (Diễn viên, Đạo diễn).

### 3.4.4 Cơ chế hợp nhất (Hybrid Integration)

Đầu ra từ ba luồng trước tiên được gộp qua cơ chế loại bỏ trùng lặp ổn định (giữ nguyên thứ tự chèn):

$$M = \text{StableDedup}(M_{\text{sem}} \cup M_{\text{con}} \cup M_{\text{col}}) \quad (3.4)$$

Tiếp theo, một bộ tiền xếp hạng tất định nhẹ (lightweight deterministic pre-ranker) theo thứ tự ưu tiên  $(r_f, y_f)$  (điểm IMDb trước, năm phát hành sau) được áp dụng để lọc sơ bộ, loại bỏ ứng viên chất lượng thấp trước khi hợp nhất thứ hạng chính thức.

Các danh sách xếp hạng từ ba luồng sau đó được hợp nhất bằng thuật toán *Weighted Reciprocal Rank Fusion* (WRRF) [26]:

$$\text{WRRF}(f) = \sum_{i \in \{\text{sem}, \text{con}, \text{col}\}} \frac{w_i}{k + \text{rank}_i(f)} \quad (3.5)$$

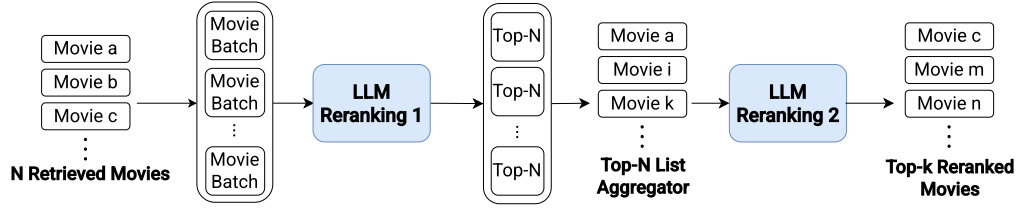
trong đó  $w_i$  là trọng số của từng luồng (thực nghiệm:  $w_{\text{sem}} = 0.4$ ,  $w_{\text{con}} = 0.35$ ,  $w_{\text{col}} = 0.25$ , được chọn để cân bằng giữa mức độ liên quan ngữ nghĩa, độ chính xác schema và đa dạng cộng tác),  $\text{rank}_i(f)$  là thứ hạng của phim  $f$  trong luồng  $i$  (phim không có mặt được gán thứ hạng ảo lớn), và  $k = 60$  là hằng số làm mượt theo chuẩn RRF.

Đáng lưu ý, các trọng số  $w_i$  được xác định thực nghiệm và *cố định trong suốt quá trình vận hành*, bất kể hình thái ngôn ngữ của truy vấn. Đây là giới hạn thiết kế sẽ được phân tích tại §3.6. Tập ứng viên cuối cùng  $M_{\text{final}}$  được sắp xếp giảm dần theo điểm WRRF và chuyển giao cho module Re-ranker.

## 3.5 Module Xếp hạng lại bằng Generative LLM

Module Re-ranker (Hình 3.6) nhận vào ba thành phần từ giai đoạn truy xuất: hồ sơ sở thích  $\hat{P}$ , tập ứng viên  $M_{\text{final}}$ , và siêu dữ liệu đã làm giàu (cốt truyện, thể loại, diễn viên, điểm số) của từng ứng viên. Cách tiếp cận này neo giữ quyết định của LLM vào thuộc tính thực chứng thay vì chỉ dựa vào ngữ cảnh hội thoại phẳng (flat dialogue context).

Để xử lý giới hạn context window, GRACE giới thiệu kỹ thuật *two-stage batching with shortlist fusion* kết hợp *metadata augmentation*: (i) giai đoạn Batch Re-ranking chia  $M_{\text{final}}$  thành các lô nhỏ, mỗi lô được LLM xếp hạng độc lập thành một shortlist; (ii) giai



Hình 3.6: Kiến trúc Module Xếp hạng lại, sử dụng cơ chế batching để xử lý tập ứng viên lớn thông qua LLM.

đoạn Final Re-ranking hợp nhất các shortlist và thực hiện một lượt đánh giá tổng hợp cuối để chọn ra top-K. Đây là kỹ thuật chưa được kết hợp đồng thời trong các hệ thống CRS trước đó như LLM-ReDial, UniCRS hay G-CRS.

Mô hình xuất ra một JSON object nghiêm ngặt chỉ chứa các phim từ tập ứng viên. Một validator độc lập kiểm tra tuân thủ schema, loại bỏ mục không hợp lệ, giải quyết trường hợp hòa (tie-breaking), và áp dụng cắt ngắn top- $k$  tất định. Cơ chế retry có giới hạn và sửa lỗi nhẹ đảm bảo hệ thống thoái hóa nhẹ nhàng (graceful degradation) thay vì thất bại hoàn toàn khi LLM trả về định dạng sai. Nhờ validator này, kết quả cuối cùng chỉ chứa các phim thuộc tập ứng viên đã được truy xuất từ KG, đảm bảo không có thực thể do LLM tự bịa ra. Giới hạn còn lại là validator chỉ kiểm tra tính hợp lệ schema của đầu ra, không đối chiếu ngược lại ràng buộc cứng của người dùng; điểm nghẽn này được giải quyết ở Chương 4.

### 3.6 Phân tích điểm nghẽn mang tính cấu trúc

Mặc dù GRACE chứng minh được hiệu quả trong việc loại bỏ ảo giác và đạt kết quả cạnh tranh trên các bộ dữ liệu chuẩn (kết quả chi tiết tại Bảng 5.2, Chương 5), kiến trúc Static Pipeline của nó bộc lộ ba điểm nghẽn có tính cấu trúc khi triển khai thực tế:

1. **Tính phi linh hoạt của Pipeline (Pipeline Rigidity):** Luồng xử lý một chiều không có khả năng tự điều chỉnh khi một bước trung gian thất bại. Ví dụ điển hình: nếu LLM trong bước tóm tắt trích xuất nhầm thể loại từ hội thoại (diễn giải "phim căng thẳng" thành *horror* thay vì *thriller*), toàn bộ tập ứng viên từ luồng Content-based sẽ lệch hướng, nhưng pipeline không có cơ chế phát hiện và sửa lỗi này.
2. **Thiếu điều phối trọng số động (Lack of Dynamic Weighting):** Trọng số WRRF

cổ định ( $w_{\text{sem}} = 0.4$ ,  $w_{\text{con}} = 0.35$ ,  $w_{\text{col}} = 0.25$ ) không phản ánh sự khác biệt về ngữ nghĩa giữa các loại truy vấn. Khi người dùng hỏi "*phim gì có cùng diễn viên với Inception?*", đây là truy vấn thuần đề thị, do đó trọng số  $w_{\text{col}}$  lẽ ra phải chiếm ưu thế tuyệt đối. Ngược lại, với yêu cầu "*phim có không khí như Blade Runner?*", truy xuất ngữ nghĩa mới là kênh hữu ích. Pipeline tính không phân biệt được hai tình huống này.

3. **Chi phí và Độ trễ từ Generative LLM:** Việc dùng Generative LLM cho toàn bộ khâu re-ranking gây ra độ trễ đáng kể trong tương tác thời gian thực. Khi tập ứng viên lớn phải qua nhiều lô đánh giá, mỗi lô đều phát sinh một lần gọi API, tích lũy cả về chi phí lẫn latency, đặc biệt bất lợi khi cần phản hồi tức thì trong hội thoại.

## Kết luận chương

Chương 3 đã phân tích chi tiết từng thành phần kỹ thuật của GRACE: từ cấu trúc KG, cơ chế tóm tắt hội thoại, ba luồng truy xuất lai, cho đến giai đoạn re-ranking bằng Generative LLM. Những thành tựu của GRACE trong việc loại bỏ ảo giác nhờ KG và cơ chế validator là cơ sở kỹ thuật quan trọng được ARGOS kế thừa. Tuy nhiên, ba điểm nghẽn được xác định (pipeline cứng nhắc, trọng số tĩnh và chi phí re-ranking) đặt ra yêu cầu rõ ràng cho kiến trúc kế tiếp: một hệ thống có khả năng tự điều chỉnh chiến lược và phản hồi nội bộ. Đây là nền tảng để Chương 4 trình bày kiến trúc đa tác tử **ARGOS**.

## Chương 4

# Kiến trúc đề xuất: Hệ thống Đa tác tử ARGOS

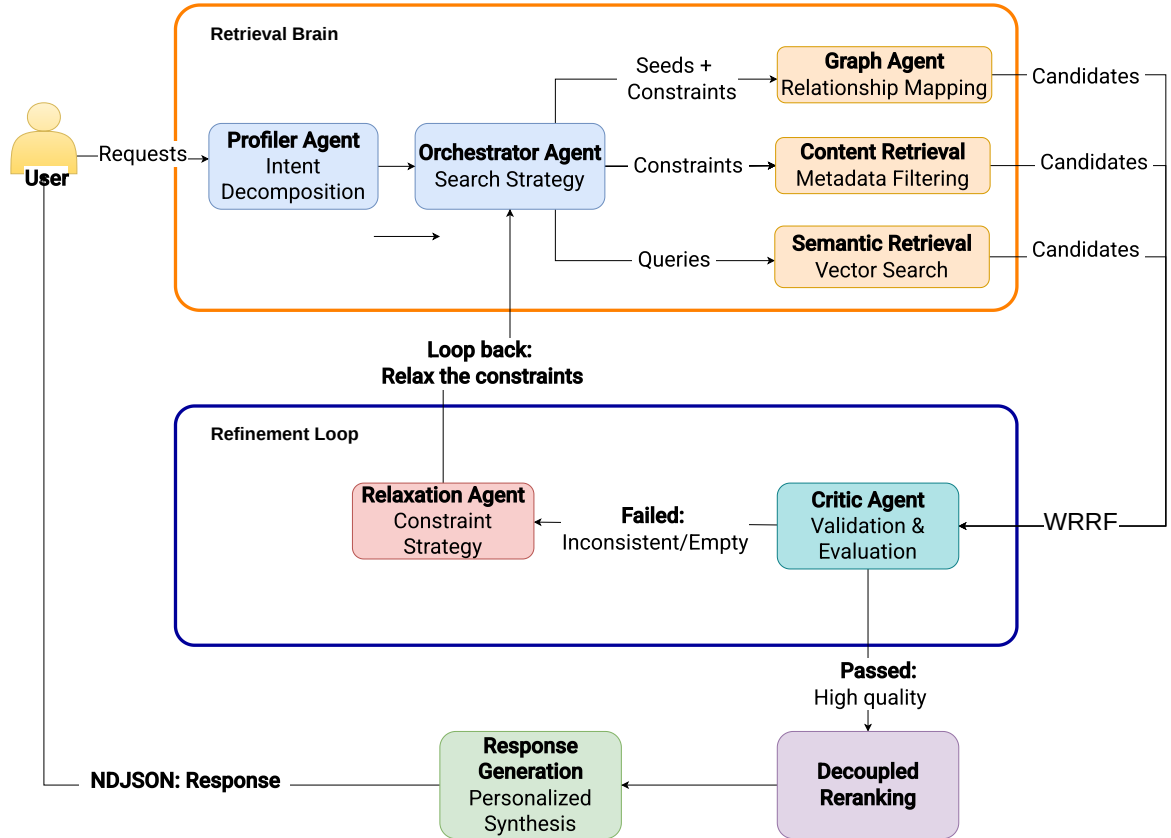
Chương này trình bày thiết kế và cơ chế vận hành của hệ thống **ARGOS** (Adaptive Reasoning and Graph Orchestration System). Các điểm nghẽn được xác định tại Chương 3 (thiếu cơ chế tự sửa lỗi, thiếu khả năng nói lỏng ràng buộc, và trọng số truy xuất cố định) được giải quyết bằng cách phân tách các chức năng thành năm tác tử chuyên biệt: Profiler, Graph Agent, Orchestrator, Critic và Relaxation Agent. Các tác tử này liên kết với nhau qua các vòng lặp phản hồi dựa trên mô hình ReAct, tạo nên một kiến trúc có khả năng tự phát hiện lỗi, điều chỉnh chiến lược truy xuất và khôi phục hội thoại khi gặp truy vấn quá khắt khe.

### 4.1 Động lực chuyển đổi sang Vòng lặp phản hồi Đa tác tử (Multi-Agent Feedback Loop)

Pipeline tuyến tính của GRACE xử lý mỗi truy vấn theo một chiều duy nhất: phân tích ý định → truy xuất → xếp hạng. Khi bước nào thất bại (truy vấn đồ thị trả về rỗng, hay toàn bộ ứng viên bị lọc bởi ràng buộc cứng), pipeline dừng lại và trả về kết quả trống mà không có cơ chế khắc phục. Sự vắng mặt của phản hồi nội bộ (internal feedback) khiến lỗi ở bước đầu lan truyền và khuếch đại qua các bước sau.

ARGOS giải quyết bằng mô hình tổ chức khác: mỗi chức năng cốt lõi được giao

cho một tác tử chuyên biệt (Profiler, Graph Agent, Orchestrator, Critic, Relaxation) và các tác tử này giao tiếp qua tín hiệu phản hồi thay vì truyền dữ liệu một chiều. Khi Critic Agent phát hiện không đủ kết quả hợp lệ, nó không trả về lỗi mà phát tín hiệu kích hoạt Relaxation Agent, tác tử này tái cấu trúc ý định và khởi động lại vòng truy xuất. Tổng quan kiến trúc được minh họa trong Hình 4.1.

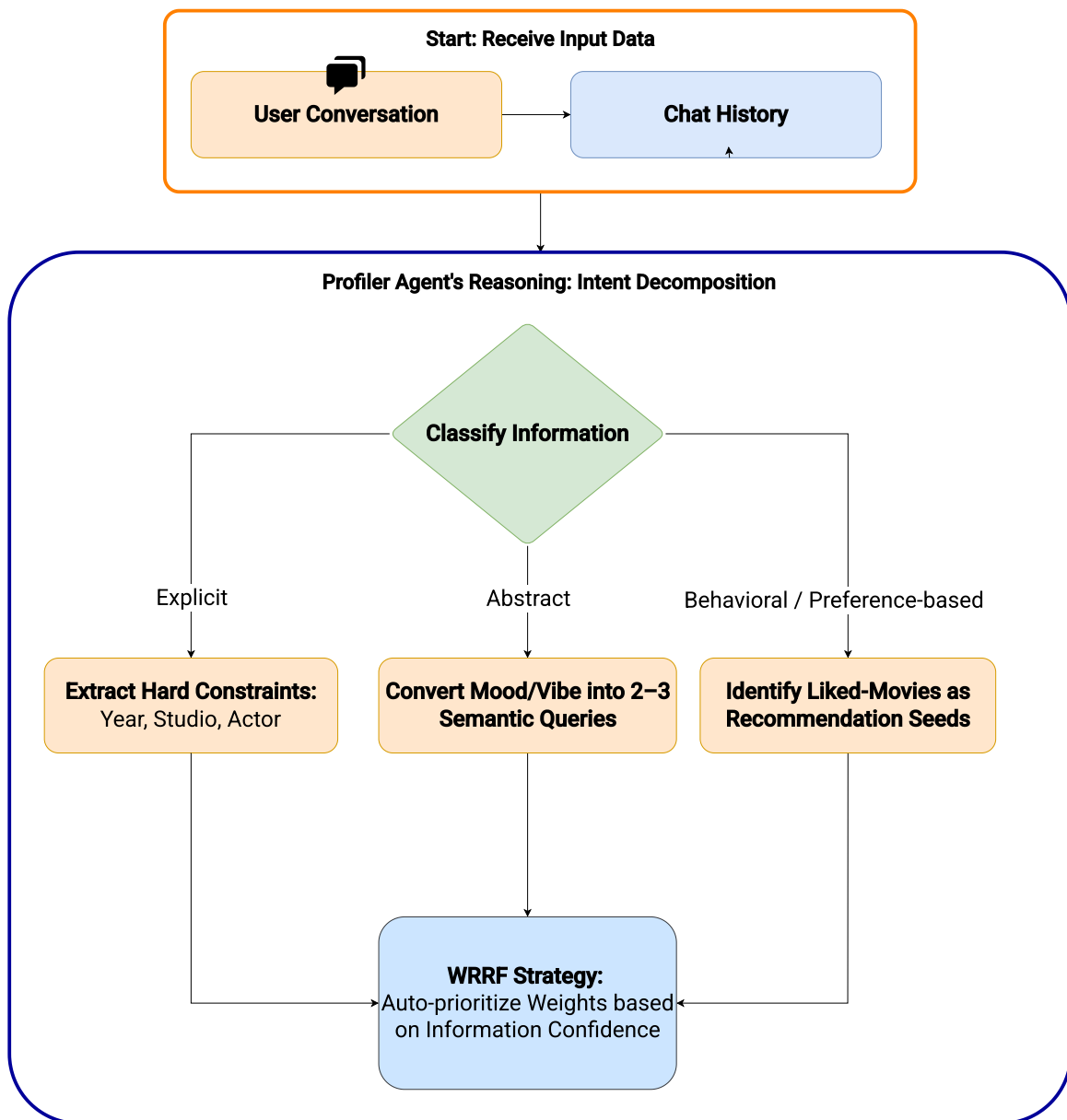


Hình 4.1: Kiến trúc tổng quan của hệ thống đa tác tử ARGOS. Khác với kiến trúc tĩnh, ARGOS đưa vào các vòng lặp phản hồi nội bộ (feedback loops) và cơ chế đánh giá lại trọng số động thông qua các Agent chuyên biệt.

## 4.2 Tác tử Hồ sơ (Profiler Agent) và Phân rã ý định

Profiler Agent là tác tử đầu tiên trong chuỗi xử lý, chịu trách nhiệm chuyển hóa yêu cầu ngôn ngữ tự nhiên của người dùng thành các cấu trúc dữ liệu chuẩn hóa (Structured Output) để các tác tử sau có thể sử dụng nhất quán. Quá trình này gồm ba cơ chế cốt lõi, được minh họa tổng quan trong Hình 4.2.

Như minh họa trong Hình 4.2, điểm cốt lõi của Profiler Agent nằm ở bước **Phân**



Hình 4.2: Luồng suy luận của Profiler Agent: phân loại thông tin đầu vào thành ba nhánh độc lập (Explicit, Abstract, Behavioral) và tổng hợp bộ trọng số WRRF.

**loại thông tin (Classify Information):** thay vì xử lý đồng nhất toàn bộ phát ngôn người dùng, tác tử nhận diện bản chất của từng mảnh thông tin và định tuyến sang pipeline phù hợp. Thông tin tường minh (Explicit) như tên diễn viên, năm phát hành hay hãng sản xuất được đóng gói thành tập Ràng buộc cứng - vừa làm điều kiện lọc Cypher cho Graph Agent, vừa là tiêu chí kiểm định của Critic Agent ở giai đoạn sau. Thông tin trừu tượng (Abstract) mang tính cảm xúc như "phim buồn" hay "không khí u ám" được diễn giải thành 2-3 câu truy vấn ngữ nghĩa độc lập nhằm khai thác tối đa không gian vector. Thông tin hành vi (Behavioral), tức các bộ phim người dùng đã xem và yêu thích, được dùng làm hạt giống cho luồng Graph Agent. Song song với quá trình phân loại thông tin, Profiler Agent phân tích đặc trưng của truy vấn để dự báo trước bộ trọng số WRRF phù hợp nhất với hình thái truy vấn hiện tại, cung cấp tín hiệu định tuyến cho Orchestrator Agent trước khi các luồng truy xuất được kích hoạt.

#### 4.2.1 Trích xuất thực thể: Ràng buộc cứng và Hạt giống gợi ý

Mục này trình bày cách Profiler Agent xử lý đồng thời hai nhánh thông tin mang tính cụ thể, xác định: nhánh **Tường minh (Explicit)** cho ra bộ Ràng buộc cứng (Hard Constraints), và nhánh **Hành vi (Behavioral)** cho ra tập Hạt giống gợi ý (Recommendation Seeds) - cả hai đều là đầu vào cốt lõi của Graph Agent ở bước sau.

**Nhánh Tường minh → Ràng buộc cứng.** Profiler Agent nhận diện và trích xuất các thực thể định danh (tên đạo diễn, diễn viên) và các tiêu chí định lượng (năm sản xuất, ngưỡng điểm đánh giá) từ ngữ cảnh hội thoại. Quá trình này không chỉ dừng lại ở nhận diện thực thể mà còn bao gồm bước **chuẩn hóa tri thức (Knowledge Normalization)** - đối soát với ngữ cảnh hội thoại để đảm bảo nhất quán trước khi đóng gói. Kết quả được hệ thống hóa thành danh sách **Ràng buộc cứng (Hard Constraints)** dưới dạng các vị từ ngôn ngữ tự nhiên:

```
"hard_constraints": [  
    "Directed by Christopher Nolan",  
    "Released between 2010 and 2020",  
    "Must not include Romance genre",  
    "IMDb rating >= 8.0"  
]
```



Việc biểu diễn ràng buộc dưới dạng chuỗi ngôn ngữ tự nhiên thay vì trường cố định mang lại tính linh hoạt quan trọng: Graph Agent có thể ánh xạ từng ràng buộc sang mệnh đề WHERE phù hợp trong Cypher nhờ khả năng hiểu ngữ nghĩa của LLM - chi tiết cơ chế này được trình bày tại Mục 4.3.1. Đáng chú ý, bộ ràng buộc cứng này còn đóng vai trò kép: được Graph Agent tiêu thụ để xây dựng truy vấn đồ thị, và được Critic Agent sử dụng ở giai đoạn sau để kiểm định toàn bộ tập ứng viên - bất kể ứng viên đó đến từ luồng nào.

**Nhánh Hành vi → Hạt giống gợi ý.** Song song với nhánh trên, Profiler Agent nhận diện các bộ phim người dùng đã đề cập tích cực trong lịch sử hội thoại (ví dụ: *"tôi rất thích John Wick"*) và đánh dấu chúng là **Hạt giống gợi ý (Recommendation Seeds)**. Khác với ràng buộc cứng (dùng để lọc), hạt giống gợi ý đóng vai trò là điểm xuất phát cho việc duyệt đồ thị: Graph Agent sử dụng chúng làm neo (anchor node) để khám phá các phim có chung diễn viên, đạo diễn hoặc ekip sáng tạo - qua đó tận dụng cấu trúc quan hệ của KG để mở rộng không gian tìm kiếm theo hướng cộng tác ẩn (implicit collaborative filtering).

## 4.2.2 Đa dạng hóa ngữ nghĩa (Intent Expansion / Multi-Query)

Trong các mô hình truy xuất ngữ nghĩa (Dense Retrieval) truyền thống, việc mã hóa toàn bộ truy vấn phức tạp của người dùng vào một vector duy nhất (single embedding vector) thường dẫn đến hiện tượng "sụp đổ ngữ nghĩa" (semantic collapse). Khi một câu lệnh chứa nhiều khái niệm trực giao (orthogonal concepts), không gian vector thường phải lấy trung bình (average out), làm phai nhòa các đặc trưng quan trọng.

Để khắc phục hạn chế này, Profiler Agent triển khai chiến lược **đa truy vấn (Multi-Query Expansion)**. Thay vì chỉ sử dụng một biểu diễn chung, tác tử phân rã ý định ban đầu thành một tập hợp các góc độ truy vấn độc lập  $Q = \{q_1, q_2, \dots, q_n\}$ . Các câu truy vấn này khai thác các đặc trưng tiềm ẩn (latent semantic features) riêng biệt như:

- **Cốt truyện và Chủ đề (Plot & Theme):** Trích xuất nội dung lõi của câu chuyện.
- **Không khí và Cảm xúc (Vibe & Mood):** Trích xuất sắc thái cảm nhận mà người dùng mong muốn.
- **Bối cảnh và Kỹ thuật điện ảnh (Setting & Cinematography):** Các yếu tố về hình

ảnh, thời đại hoặc phong cách.

Diễn hình, một yêu cầu phức tạp như "Tôi muốn xem một phim giống John Wick, bối cảnh Châu Á nhưng mang màu sắc buồn bã" sẽ không bị nén thành một vector hỗn độn. Thay vì đó, Profiler phân giải nó thành:  $q_1$  = "hành động nhịp độ cao, sát thủ",  $q_2$  = "bối cảnh Nhật Bản hoặc Hồng Kông", và  $q_3$  = "tâm lý nặng nề, kết cục bi thảm".

Quá trình này giúp mở rộng không gian tìm kiếm đa chiều (multi-dimensional search space), gia tăng đáng kể tỷ lệ bao phủ (Recall) của hệ thống. Cần lưu ý rằng, mặc dù việc phân rã ý định có vẻ giống một phép toán hợp (OR) tại bước truy xuất để tối ưu độ phủ, tính hội tụ và điều kiện giao (AND) của các yêu cầu vẫn được đảm bảo chặt chẽ ở các giai đoạn sau thông qua thuật toán hợp nhất thứ hạng (Fusion) và bước đối soát của Critic Agent.

### 4.2.3 Dự báo chiến lược trọng số động (Dynamic Weighting / Adaptive Routing)

Trong kiến trúc tĩnh GRACE, việc hợp nhất kết quả từ nhiều luồng dữ liệu (Semantic, Content, Graph) phụ thuộc vào một bộ siêu tham số (hyperparameters) tĩnh. Tuy nhiên, mỗi truy vấn lại yêu cầu một cấu hình khác nhau. Ví dụ, câu hỏi "Phim nào của Christopher Nolan đạo diễn?" mang đậm tính cấu trúc, trong khi câu "Phim nào khiến tôi phải suy ngẫm về nhân sinh?" lại mang đậm tính ngữ nghĩa.

Sau khi hoàn tất bước phân loại thông tin, Profiler Agent tổng hợp kết quả từ cả ba nhánh để xác định bộ trọng số WRRF phù hợp. Nguyên tắc cốt lõi là **ưu tiên trọng số theo độ tin cậy của thông tin (Information Confidence)**: nhánh nào trích xuất được nhiều thông tin chất lượng cao thì luồng truy xuất tương ứng được khuếch đại. Dựa trên đó, Profiler Agent dự báo một vector phân bố trọng số  $w(C)$  cho các luồng truy xuất theo ngữ cảnh hội thoại  $C$ :

$$w(C) = (w_{\text{sem}}, w_{\text{con}}, w_{\text{col}})^T, \quad \text{với} \quad \sum_{i \in \{\text{sem}, \text{con}, \text{col}\}} w_i = 1, \quad w_i \in [0, 1] \quad (4.1)$$

Trong đó:

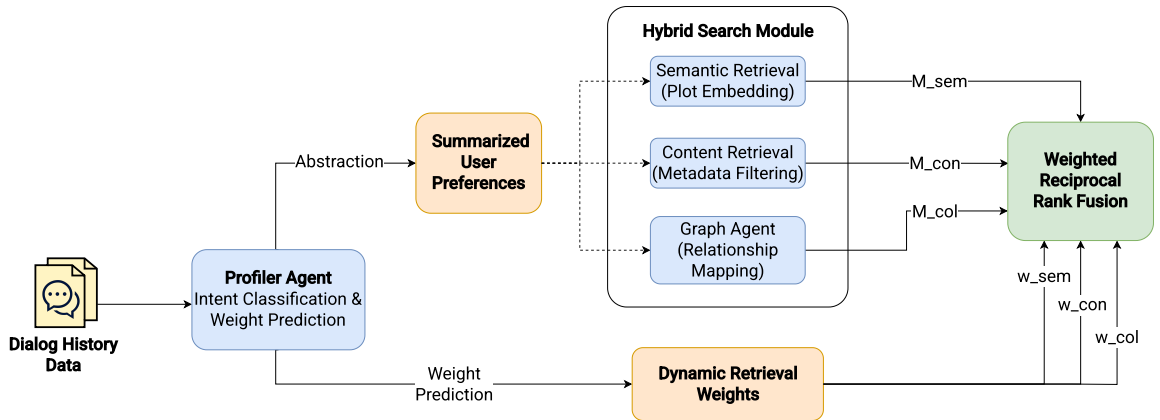
- $w_{\text{sem}}$  (Trọng số Ngữ nghĩa - Semantic Weight): Tăng cao khi truy vấn chứa nhiều

tính từ mô tả, cảm xúc, hoặc so sánh tương đồng ("giống phim A nhưng buồn hơn").

- $w_{con}$  (Trọng số Nội dung - Content Weight): Được ưu tiên khi người dùng chỉ định rõ các tiêu chí lọc cứng như thể loại, khoảng năm phát hành, hoặc ngưỡng điểm đánh giá (IMDb/Rating).
- $w_{col}$  (Trọng số Đồ thị - Graph Weight): Được kích hoạt khi truy vấn chỉ định rõ các thực thể danh định (Entities) như tên diễn viên, đạo diễn, hãng phim, hoặc khi cần duyệt Đồ thị tri thức từ các phim người dùng đã thích để mở rộng tìm kiếm.

Việc định tuyến linh hoạt (Adaptive Routing) này giúp ARGOS tự động điều phối tài nguyên tìm kiếm, tối ưu hóa sự kết hợp của nhiều nguồn tri thức mà không cần sự can thiệp thủ công của kỹ sư vào mã nguồn. Cơ chế này được minh họa cụ thể trong Hình 4.3.

**Cơ chế triển khai kỹ thuật.** LLM được nhắc bằng một prompt có cấu trúc cố định, nhận đầu vào là hồ sơ sở thích  $\hat{P}$  và kết quả phân loại ba nhánh (Explicit, Abstract, Behavioral), sau đó sinh ra một JSON object với ba trường số thực  $\{sem, con, col\}$ . Bước hậu xử lý (post-processing) chuẩn hóa bắt buộc bằng phép chia tổng để đảm bảo  $\sum w_i = 1$  bất kể giá trị thô LLM trả về. Nếu đầu ra JSON không hợp lệ hoặc chứa giá trị âm, hệ thống tự động quay về bộ trọng số mặc định  $(w_{sem}, w_{con}, w_{col}) = (0.40, 0.30, 0.30)$  và ghi log cảnh báo, đảm bảo hệ thống thoái hóa nhẹ nhàng (graceful degradation) mà không gián đoạn luồng hội thoại.



Hình 4.3: Cơ chế dự báo trọng số động (Dynamic Weight Prediction) do Profiler Agent thực hiện, cung cấp tín hiệu định tuyến cho các luồng truy xuất.

## 4.3 Tác tử Đồ thị (Graph Agent) với Vòng lặp ReAct

Graph Agent là luồng truy xuất chuyên biệt nhất trong kiến trúc ARGOS, chịu trách nhiệm tương tác trực tiếp với cơ sở dữ liệu đồ thị Neo4j. Tác tử tiếp nhận hai loại đầu vào có vai trò khác nhau: (1) **Ràng buộc cứng** từ Profiler Agent - được dùng để xây dựng các mệnh đề lọc WHERE trong Cypher, giới hạn không gian tìm kiếm theo yêu cầu tường minh; và (2) **Hạt giống gợi ý** (tên phim người dùng đã thích) - được dùng làm điểm neo (anchor) để duyệt đồ thị đa bước (multi-hop traversal), khám phá các thực thể liên quan qua chuỗi quan hệ. Hai loại đầu vào này tương ứng với hai chiến lược truy xuất song song: *lọc theo điều kiện* và *mở rộng theo cấu trúc quan hệ*.

### 4.3.1 Nhận thức cấu trúc dữ liệu (Schema-Aware Intelligence)

Điểm yếu thường thấy khi giao tiếp tự do giữa LLM và cơ sở dữ liệu đồ thị là LLM có xu hướng “ảo giác” ra các nút (nodes) hoặc các mối quan hệ (edges) không hề tồn tại trong cơ sở dữ liệu thực tế. Ví dụ, LLM có thể sinh ra các câu truy vấn đồ thị sai lệch về cấu trúc nút, trong khi đồ thị thực tế chỉ cho phép những liên kết nhất định giữa các thực thể.

Để giải quyết vấn đề này, Graph Agent được trang bị cơ chế **Nhận thức cấu trúc dữ liệu (Schema-Aware Intelligence)**: toàn bộ lược đồ KG được nhúng trực tiếp vào system prompt của tác tử ở mỗi lượt suy luận. Lược đồ này khai báo đầy đủ các nhãn nút, thuộc tính và kiểu quan hệ hợp lệ trong hệ thống:

Nodes:

```
[Film]          -> movieId, title, plot, year, runtime, imdbRating
[Director]      -> name
[Actor]         -> name
[Genre]         -> name
[ReleaseYear]  -> value
```

Relationships:

```
(:Actor)-[:ACTED_IN]->(:Film)
(:Director)-[:DIRECTED]->(:Film)
(:Film)-[:IN_GENRE]->(:Genre)
```

(:Film)-[:RELEASED\_IN]->(:ReleaseYear)

Nhờ lược đồ này, LLM có thể ánh xạ chính xác các ràng buộc cứng (nhận từ Profiler Agent dưới dạng chuỗi ngôn ngữ tự nhiên) sang mệnh đề WHERE phù hợp trong Cypher. Hai ví dụ minh họa quá trình suy luận này:

```
-- "Released between 2010 and 2020"
MATCH (f:Film)-[:RELEASED_IN]->(y:ReleaseYear)
WHERE y.value >= 2010 AND y.value <= 2020

-- "Directed by Christopher Nolan"
MATCH (d:Director)-[:DIRECTED]->(f:Film)
WHERE toLower(d.name) = 'christopher nolan'
```

Cơ chế ràng buộc qua lược đồ đảm bảo tác tử không thể sinh ra các quan hệ hoặc thuộc tính không tồn tại trong đồ thị thực tế.

### 4.3.2 Vòng lặp Suy luận và Hành động (Reasoning + Acting)

Đặc điểm cốt lõi tạo nên sự khác biệt của Graph Agent là việc tích hợp mô hình suy luận ReAct (Reasoning and Acting) [3]. Khác với các hệ thống chuyển đổi ngôn ngữ sang truy vấn đồ thị (Text-to-Graph Query) thụ động, tác tử vận hành dựa trên một chuỗi các trạng thái động được định nghĩa hình thức tại bước  $t$  như sau:

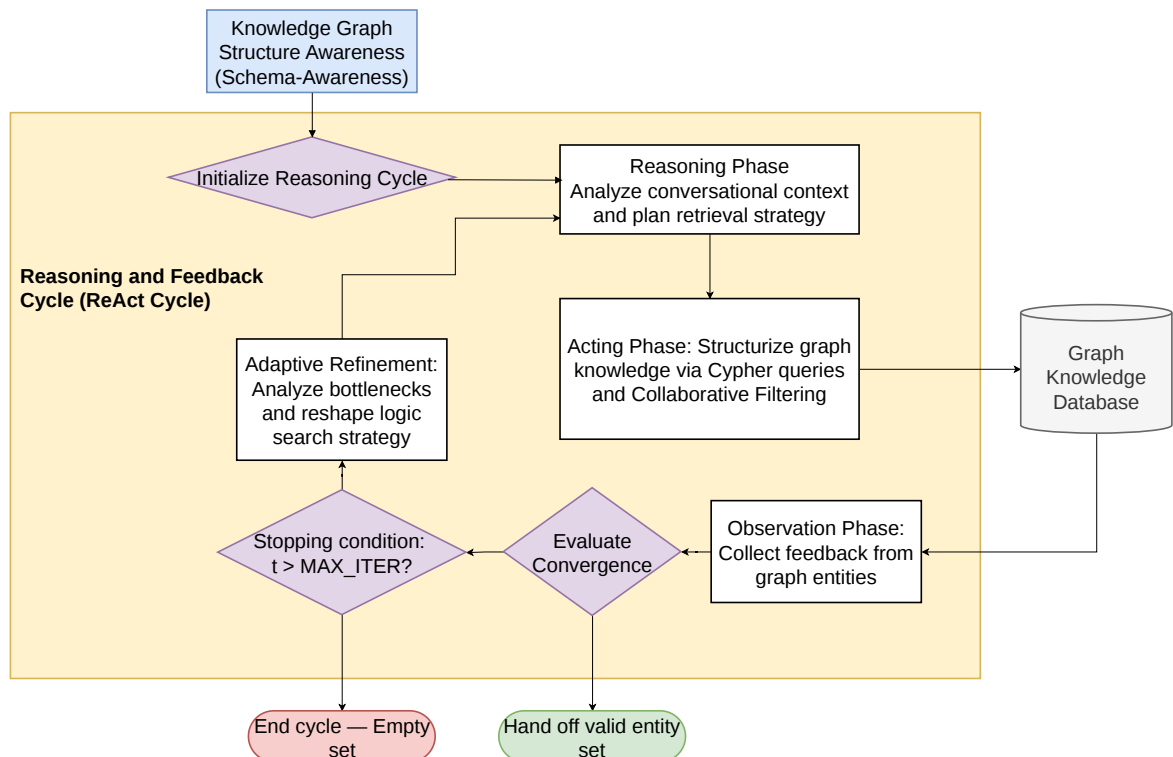
$$\text{Thought}_t \rightarrow \text{Action}_t(\text{Graph Query}) \rightarrow \text{Observation}_t(\text{DB Result}) \quad (4.2)$$

Trong chu trình tối đa 3 lượt (turns), hệ thống tiến hành tuần tự các bước:

1. **Thought:** Tác tử phân tích đồng thời hai nguồn đầu vào để lập kế hoạch chiến lược: (1) các ràng buộc cứng và lược đồ KG - xác định mệnh đề WHERE cần xây dựng; (2) tập hạt giống gợi ý (liked\_movies) - quyết định có cần kích hoạt Collaborative Traversal hay không (nếu có hạt giống, lên kế hoạch duyệt theo quan hệ ACTED\_IN hoặc DIRECTED từ phim neo). Nếu lượt trước trả về kết quả rỗng, tác tử xác định ràng buộc nào quá khắt khe và đề xuất hướng nói lỏng cho lượt tiếp theo.

2. **Action:** Dựa trên Thought, tác tử thực hiện song song hai chiến lược truy vấn. Với thông tin tường minh, tác tử sinh câu lệnh Cypher hợp lệ theo lược đồ KG, đặt các ràng buộc cứng vào mệnh đề WHERE để lọc chính xác ngay tại tầng đồ thị. Với hạt giống gợi ý (liked\_movies), tác tử thực hiện **duyet đồ thị cộng tác (Collaborative Traversal)**: khởi động từ phim neo (anchor node) và mở rộng theo quan hệ ACTED\_IN hoặc DIRECTED để khai phá các phim liên kết qua ekip sáng tạo chung.
3. **Observation:** Tác tử thu nhận trực tiếp phản hồi từ Neo4j - có thể là tập kết quả hợp lệ, thông báo lỗi cú pháp Cypher, hoặc danh sách rỗng. Phản hồi này trở thành đầu vào cho Thought ở lượt tiếp theo, khép kín vòng lặp tự sửa lỗi (Self-Correction).

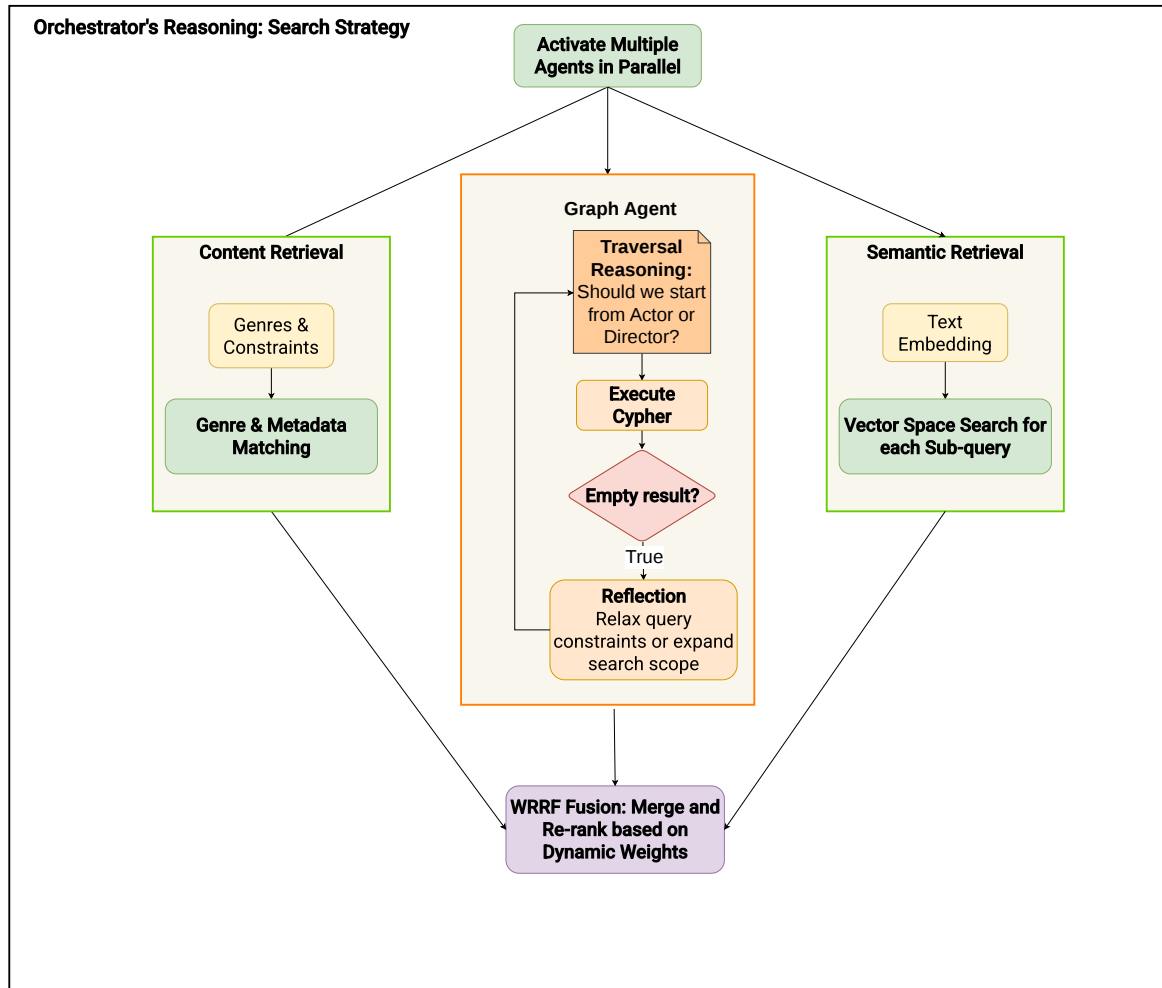
Cơ chế tự sửa lỗi (Self-Correction) này giúp hệ thống xử lý được các tình huống sai lệch cấu trúc đồ thị (Schema Mismatch) mà không cần lập trình viên viết thêm logic xử lý ngoại lệ cho từng trường hợp cụ thể. Quy trình này được mô phỏng chi tiết trong Hình 4.4.



Hình 4.4: Chu trình ReAct của Graph Agent: xen kẽ Suy luận - Hành động - Quan sát để tự sửa lỗi và tối ưu chiến lược truy vấn đồ thị.

## 4.4 Trung tâm Điều phối (Orchestrator Agent)

Orchestrator Agent hoạt động như một tác tử điều phối trung tâm, chịu trách nhiệm quản lý tài nguyên và tổng hợp thứ hạng. Hình 4.5 minh họa luồng tư duy và chiến lược tìm kiếm đa luồng của Orchestrator.



Hình 4.5: Chiến lược tìm kiếm đa luồng của Orchestrator Agent. Hệ thống kích hoạt song song ba luồng độc lập: Vòng lặp ReAct của Graph Agent, Semantic Retrieval và Content Retrieval, sau đó hợp nhất kết quả thông qua thuật toán WRRF Fusion có sử dụng trọng số động.

#### 4.4.1 Thực thi truy xuất bất đồng bộ (Asynchronous Parallel Execution)

Khác với mô hình đường ống tuần tự (sequential pipeline), Orchestrator được thiết kế để tối thiểu hóa độ trễ (latency) thông qua kiến trúc lập trình bất đồng bộ (asynchronous programming). Ngay khi Profiler Agent hoàn tất việc phân rã ý định, Orchestrator sẽ kích hoạt đồng thời ba luồng truy xuất độc lập:

- **Semantic Retrieval:** Tìm kiếm trên không gian vector mật độ cao (Dense Vector DB) để nắm bắt ý nghĩa tiềm ẩn.
- **Content Retrieval:** Khớp lệnh chính xác (Exact Match) trên cơ sở dữ liệu quan hệ hoặc hệ thống tìm kiếm văn bản để lọc theo các siêu dữ liệu cứng (metadata).
- **Graph Agent:** Thực thi vòng lặp ReAct trên cơ sở dữ liệu đồ thị để tìm kiếm các thực thể liên kết đa bước.

Quá trình xử lý song song này đảm bảo hệ thống có thể quét qua toàn bộ mặt phẳng dữ liệu khổng lồ mà không làm tăng thời gian phản hồi tổng thể (Time-To-First-Token). Điểm nghẽn về tốc độ sẽ chỉ phụ thuộc vào luồng phản hồi chậm nhất (thường là Graph Agent), thay vì tổng thời gian của cả ba luồng cộng lại.

#### 4.4.2 Hợp nhất thứ hạng với thuật toán WRRF (Dynamic WRRF)

Mỗi luồng truy xuất sẽ trả về một danh sách các ứng viên độc lập với thang điểm (score scale) hoàn toàn khác nhau (ví dụ: điểm Cosine Similarity của cơ sở dữ liệu Vector khác biệt hoàn toàn với điểm số cấu trúc của đồ thị). Để đồng bộ hóa, Orchestrator áp dụng thuật toán Hợp nhất Thứ hạng Nghịch đảo có Trọng số (Weighted Reciprocal Rank Fusion - WRRF).

Cho  $R_m(i)$  là thứ hạng của vật phẩm  $i$  trong danh sách trả về của phương pháp truy xuất  $m \in \{\text{sem}, \text{con}, \text{col}\}$ , điểm số cuối cùng của vật phẩm  $i$  được tính bằng:

$$\text{WRRF\_Score}(i) = \sum_m \frac{w_m}{k + R_m(i)} \quad (4.3)$$

Trong đó:



- $w_m$  là trọng số của phương pháp  $m$  do Profiler Agent dự báo theo ngữ cảnh hội thoại tại mỗi lượt. Đây là khác biệt cốt lõi so với GRACE, nơi bộ trọng số được cố định tại  $(w_{\text{sem}}, w_{\text{con}}, w_{\text{col}}) = (0.40, 0.35, 0.25)$  cho toàn bộ hệ thống.
- $k$  là một hằng số làm mượt (smoothing constant), thường được đặt bằng 60 để ngăn chặn các vật phẩm ở top 1 chiếm ưu thế quá mức.

Cơ chế WRRF vinh danh các ứng viên xuất hiện ở phần giao tập hợp (intersection) của nhiều phương pháp tìm kiếm (nghĩa là vừa có ngữ nghĩa tốt, vừa có liên kết đồ thị mạnh). Điều này giải quyết bài toán "điều kiện hợp" (OR) của Multi-Query: một vật phẩm chỉ thỏa mãn  $q_1$  sẽ có thứ hạng thấp hơn nhiều so với vật phẩm thỏa mãn đồng thời cả  $\{q_1, q_2, q_3\}$ . Do đó, WRRF đóng vai trò như một phép toán "giao mềm" (soft AND), định hướng sự hội tụ về phía các kết quả thỏa mãn toàn diện ý định người dùng. Việc áp dụng trọng số động  $w_m$  giúp khuếch đại ảnh hưởng của luồng dữ liệu phù hợp nhất với loại truy vấn, từ đó tạo ra một danh sách ứng viên ưu tú mang tính thích ứng cao để chuyển giao cho pha kiểm định.

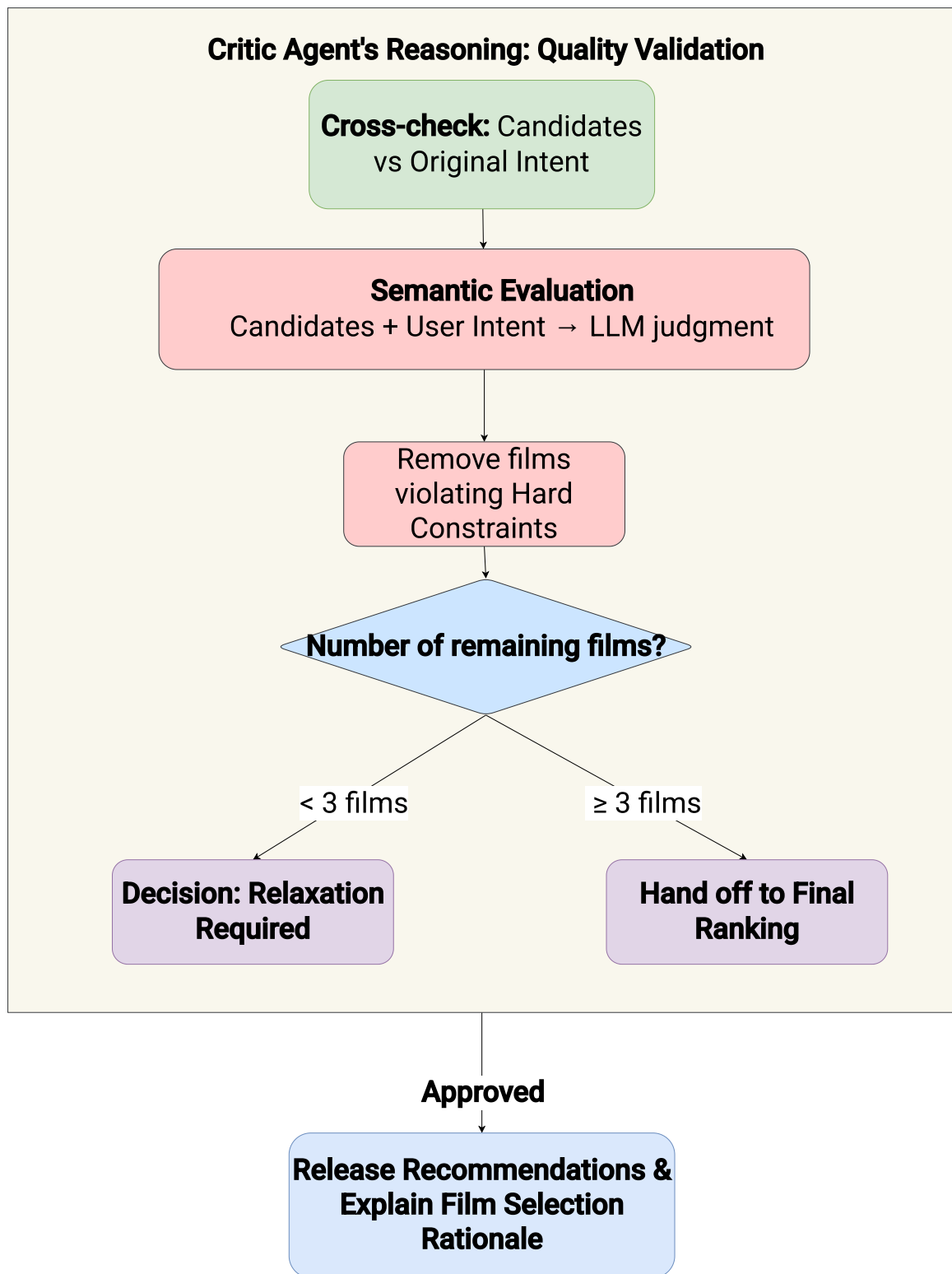
## 4.5 Tác tử Kiểm định (Critic Agent) - Kiểm soát chất lượng

Tác tử kiểm định (Critic Agent) đóng vai trò là một cơ chế rà soát độc lập (Quality Gate) trong kiến trúc ARGOS, hoạt động như một đối trọng để kiểm soát tính hợp lệ của danh sách ứng viên do các module truy xuất cung cấp. Cơ chế hoạt động của Critic Agent được minh họa trong Hình 4.6.

### 4.5.1 Đối soát chéo và triệt tiêu ảo giác (Anti-Hallucination)

Hệ thống tiền đề GRACE đã giải quyết ảo giác ở cấp độ thực thể: mọi ứng viên đều truy xuất từ KG nên không có phim "tưởng tượng" nào lọt vào tập kết quả. Tuy nhiên, GRACE chưa có cơ chế đối chiếu từng ứng viên với các ràng buộc cứng mà người dùng đặt ra trong hội thoại. Với kiến trúc tĩnh không có vòng lặp phản hồi, nếu tập ứng viên hợp lệ sau lọc quá ít hoặc rỗng, pipeline dừng lại mà không có cơ chế khắc phục.

Trong ARGOS, vấn đề này còn phức tạp hơn do kiến trúc đa luồng. Graph Agent lọc



Hình 4.6: Cơ chế hoạt động của Critic Agent. Tác tử thực hiện đối soát chéo các ứng viên với Ràng buộc cứng (Hard Constraints) và kích hoạt cơ chế phản hồi (Feedback Trigger) nếu số lượng ứng viên hợp lệ không đạt ngưỡng dung sai.

trực tiếp bằng Cypher (WHERE) nên ứng viên của luồng này đã thỏa mãn ràng buộc. Nhưng hai luồng còn lại có mức độ nhận thức về ràng buộc rất hạn chế: Semantic Retrieval hoàn toàn không có knowledge về ràng buộc cứng, trả về ứng viên thuần túy theo độ tương đồng vector; Content Retrieval chỉ lọc theo trường genres có cấu trúc riêng từ Profiler output - nó không parse các ràng buộc về năm phát hành, đạo diễn hay điểm đánh giá vốn được biểu diễn dưới dạng chuỗi ngôn ngữ tự nhiên trong `hard_constraints`. Sau khi Orchestrator hợp nhất ba luồng bằng WRRF, tập ứng viên kết quả có thể chứa các phim vi phạm ràng buộc từ hai luồng không được lọc đầy đủ đó. **Critic Agent vì vậy đóng vai trò kiểm tra xuyên luồng (cross-stream verification)**: đảm bảo mọi ứng viên, bất kể nguồn gốc, đều thỏa mãn toàn bộ ràng buộc tường minh trước khi tiến vào giai đoạn xếp hạng.

Để thực hiện vai trò kiểm tra xuyên luồng này, Critic Agent không kiểm tra từng thuộc tính theo luật cứng nhắc mà thực hiện **một lượt đánh giá ngữ nghĩa thống nhất**: toàn bộ danh sách ứng viên (tiêu đề, cốt truyện) cùng với tóm tắt ý định người dùng được đưa vào LLM để phán quyết tập thể. LLM lập luận dựa trên ngữ cảnh hội thoại để xác định các ứng viên vi phạm ràng buộc một cách rõ ràng - chẳng hạn phim kinh dị lọt vào khi người dùng yêu cầu phim hoạt hình cho trẻ em, hoặc phim của đạo diễn sai so với yêu cầu tường minh. Cách tiếp cận này có hai ưu điểm so với kiểm tra Boolean theo trường dữ liệu:

- **Linh hoạt với ngôn ngữ tự nhiên**: Ràng buộc không cần ánh xạ sang trường cụ thể trong cơ sở dữ liệu; LLM tự suy luận vi phạm từ mô tả ngữ nghĩa của ứng viên.
- **Bảo toàn ứng viên biên**: Các phim có thể phù hợp nhưng không chắc chắn được giữ lại thay vì bị loại do quy tắc quá chặt, tránh làm rỗng tập kết quả.

Những ứng viên vi phạm ràng buộc một cách hiển nhiên bị loại khỏi tập kết quả, bảo toàn độ chuẩn xác (Precision) của hệ thống trước khi chuyển sang giai đoạn xếp hạng.

#### 4.5.2 Kích hoạt phản hồi và Lập báo cáo lỗi (Feedback Trigger & Error Reporting)

Sau khi sàng lọc qua bộ Ràng buộc cứng, hệ thống thu được tập hợp ứng viên hợp lệ  $M_{\text{valid}}$ . Critic Agent thực hiện kiểm tra điều kiện ngắt quãng dựa trên ngưỡng dung sai  $\tau$

(thường thiết lập  $\tau = 3$  để đảm bảo tập kết quả đủ đa dạng cho giai đoạn xếp hạng tiếp theo):

$$\text{Trigger} = \begin{cases} \text{Accept}, & \text{nếu } |M_{\text{valid}}| \geq \tau \\ \text{Relaxation\_Required}, & \text{nếu } |M_{\text{valid}}| < \tau \end{cases} \quad (4.4)$$

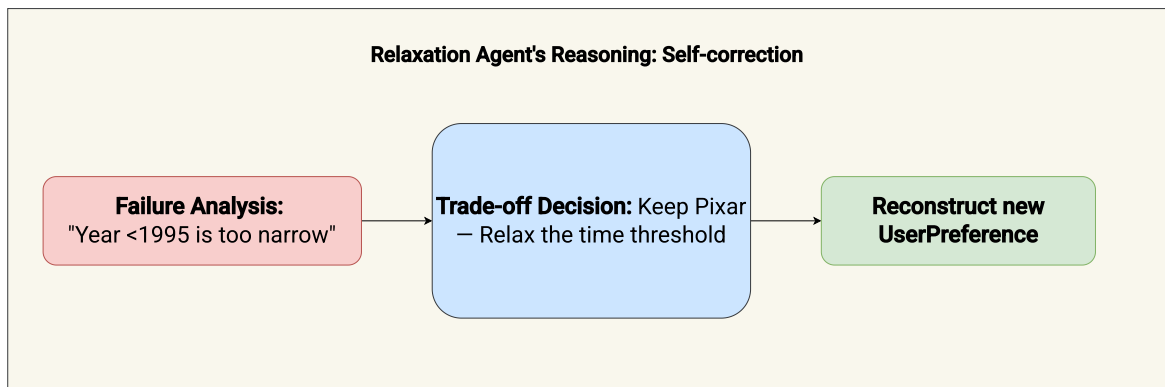
Nếu tín hiệu Relaxation\_Required được phát ra, thay vì trả về kết quả rỗng một cách thụ động, Critic Agent đồng thời sinh ra một **báo cáo lý luận** (*critic\_reasoning*) bằng ngôn ngữ tự nhiên, giải thích rõ tại sao các ứng viên bị loại và gợi ý hướng nối lỏng phù hợp. Ví dụ: “*Tìm thấy 5 phim hành động của đạo diễn Nolan, nhưng không có phim nào phát hành sau năm 2020 - cần nối lỏng ràng buộc năm phát hành.*” Báo cáo này được chuyển trực tiếp sang Relaxation Agent, định hướng chính xác cho tác tử biết cần điều chỉnh ràng buộc nào thay vì phải suy luận lại từ đầu.

## 4.6 Tác tử Thỏa hiệp (Relaxation Agent) - Cơ chế tự sửa lỗi

Trong kiến trúc đa tác tử, kịch bản tập kết quả rỗng do ràng buộc quá nghiêm ngặt là một thất bại cần được giải quyết ở tầng thiết kế, không phải bằng thông báo lỗi ở tầng giao diện. Relaxation Agent hiện thân cho triết lý **thoái hóa nhẹ nhàng (graceful degradation)**: thay vì dừng lại khi không tìm được kết quả khớp hoàn toàn, hệ thống chủ động thương lượng với ràng buộc - bảo toàn cốt lõi ý định và nhường nhịn những gì có thể nhường. Hình 4.7 minh họa quy trình hai bước của tác tử này.

### 4.6.1 Phân tích điểm nghẽn và Phân cấp ưu tiên (Sacrifice Hierarchy)

Relaxation Agent tiếp nhận ba đầu vào từ vòng lặp thất bại: hồ sơ sở thích đang hoạt động (*user\_preferences*), danh sách ràng buộc cứng (*hard\_constraints*), và báo cáo phân tích thất bại (*critic\_reasoning*) từ Critic Agent. *critic\_reasoning* là điểm xuất phát quan trọng vì nó đã chỉ rõ ràng buộc nào đang gây tắc nghẽn - Relaxation Agent không phải suy diễn lại từ đầu mà chỉ cần xác định *có thể* điều chỉnh ràng buộc đó theo hướng nào.



Hình 4.7: Quy trình phân tích điểm nghẽn và tái cấu trúc ý định của Relaxation Agent. Dựa trên phân cấp ưu tiên (Sacrifice Hierarchy), tác tử nới lỏng các điều kiện tìm kiếm và khởi động lại vòng lặp hội thoại mới.

Đây là lúc cơ chế **Phân cấp ưu tiên (Sacrifice Hierarchy)** được kích hoạt. Không phải mọi ràng buộc đều có thể thỏa hiệp như nhau: một ràng buộc người dùng nhấn mạnh nhiều lần trong hội thoại có chi phí thỏa hiệp cao hơn nhiều so với một điều kiện được đề cập thoáng qua. LLM đọc lại toàn bộ lịch sử hội thoại, suy luận về mức độ gắn kết này và xếp hạng từng ràng buộc vào ba tầng:

- **Cốt lõi (Core) - bất khả xâm phạm:** Thể loại và các yêu cầu nội dung tường minh (ví dụ: "phim không có cảnh bạo lực cho trẻ em") phản ánh nhu cầu giải trí căn bản hoặc giới hạn phân loại nội dung không thể đàm phán. Hệ thống tuyệt đối không được đánh đổi tầng này.
- **Thứ cấp (Soft) - ứng viên nới lỏng đầu tiên:** Các mốc thời gian chính xác hoặc điều kiện giao của nhiều thực thể thường mang tính tham khảo hơn là bắt buộc. Ví dụ: "*Phim Kinh dị sản xuất đúng năm 1950*" có thể mở rộng thành "*Phim Kinh dị thập niên 1950*"; hay ràng buộc "*phim hành động với cả Tom Cruise và Brad Pitt*" có thể điều chỉnh thành "*phim hành động với Tom Cruise hoặc Brad Pitt*".
- **Tùy chọn (Optional) - có thể loại bỏ:** Các yêu cầu về hãng sản xuất, quốc gia, hoặc phong cách kỹ thuật điện ảnh thường đóng vai trò định hướng chứ không phải điều kiện lọc cứng, và có thể được bỏ qua hoàn toàn nếu cần.

Nguyên tắc chỉ đạo xuyên suốt là **nới lỏng tối thiểu (minimal relaxation)**: chỉ điều chỉnh đúng ràng buộc gây tắc nghẽn, không cào bằng toàn bộ bộ lọc. Cách tiếp cận này

bảo toàn tối đa ý định ban đầu của người dùng, đảm bảo kết quả trả về dù không hoàn hảo vẫn là phiên bản gần nhất với điều họ thực sự muốn.

#### 4.6.2 Tái cấu trúc ý định (Intent Reconstruction)

Sau khi thiết lập phương án thỏa hiệp, Relaxation Agent tiến hành **Tái cấu trúc ý định (Intent Reconstruction)**: tổng hợp kết quả phân tích thành một hồ sơ thích mới hoàn toàn tương thích với lược đồ đầu ra của Profiler Agent. Sự tương thích này là thiết kế có chủ đích - Orchestrator có thể tiêu thụ đầu ra của Relaxation Agent theo cùng một giao thức như tiêu thụ đầu ra ban đầu của Profiler Agent, không cần bước chuyển đổi định dạng nào ở giữa. Hai trường được cập nhật trực tiếp:

1. **hard\_constraints**: Phiên bản nói lỏng của danh sách ràng buộc - ràng buộc gây tắc nghẽn được thay thế bằng biến thể rộng hơn theo phán quyết của Sacrifice Hierarchy, trong khi mọi ràng buộc cốt lõi được giữ nguyên vẹn.
2. **semantic\_queries**: Tập truy vấn ngữ nghĩa được viết lại để phản ánh không gian tìm kiếm mở rộng tương ứng, đảm bảo luồng Semantic Retrieval cũng hướng tới cùng vùng ứng viên mới với luồng Graph Agent.

Hồ sơ tái cấu trúc được đẩy trở lại Orchestrator Agent, kích hoạt Attempt 2 trên không gian ràng buộc đã nói lỏng. Điều quan trọng là thông tin về sự thỏa hiệp không bị giấu đi: nếu Attempt 2 thành công, module sinh phản hồi cuối cùng sẽ nhận được đầy đủ ngữ cảnh về sự thay đổi ràng buộc để tạo ra một *diễn giải minh bạch (transparent explanation)* đi kèm với gợi ý, ví dụ: "*Không tìm thấy phim hoạt hình Pixar nào trước năm 1995, nhưng đây là những tác phẩm xuất sắc của Studio Ghibli trong cùng thời kỳ mang tinh thần tương tự.*"

Cơ chế này hiện thực hóa triết lý **tự phục hồi có giải thích (explainable self-healing)**: ARGOS không chỉ giải cứu hội thoại khỏi bế tắc mà còn đồng hành cùng người dùng qua từng bước điều chỉnh, duy trì niềm tin vào chất lượng gợi ý ngay cả khi điều kiện lý tưởng không được đáp ứng.

## 4.7 Cải tiến Module Xếp hạng hai giai đoạn (Decoupled Reranking)

Khi Critic Agent xác nhận tập ứng viên hợp lệ đủ ngưỡng  $\tau$ , luồng xử lý tiến vào giai đoạn xếp hạng tinh tế. Nhiệm vụ của giai đoạn này là thu gọn tập ứng viên - có thể lên đến hàng trăm phim sau bước hợp nhất WRRF - xuống danh sách Top- $K$  cuối cùng để trình bày cho người dùng, đồng thời bảo toàn chất lượng xếp hạng.

GRACE đã áp dụng cơ chế xếp hạng hai giai đoạn theo lô (two-stage batching) sử dụng Generative LLM cho cả hai giai đoạn, cho phép xử lý danh sách ứng viên lớn nhưng chi phí tính toán tăng nhanh khi số lượng ứng viên tăng. ARGOS kế thừa cấu trúc hai giai đoạn này nhưng thay thế LLM ở giai đoạn lọc đầu bằng Cross-Encoder chuyên dụng (SLM) để tối ưu tốc độ, chỉ dùng Generative LLM ở giai đoạn suy luận cuối.

Sự cần thiết của phương pháp này xuất phát từ hai hạn chế cốt lõi của Generative LLM:

1. **Hiện tượng Lost-in-the-Middle:** Khi đưa một danh sách dài hàng trăm ứng viên vào ngữ cảnh (context window) của LLM, mô hình thường chỉ tập trung vào các kết quả ở đầu và cuối văn bản, bỏ qua các thông tin quan trọng ở giữa.
2. **Độ phức tạp tính toán (Computational Complexity):** Việc áp dụng cơ chế Self-Attention trên toàn bộ chuỗi văn bản của hàng trăm siêu dữ liệu phim sẽ dẫn đến phức tạp tính toán bậc hai ( $O(N^2)$ ), tăng đáng kể chi phí API và độ trễ.

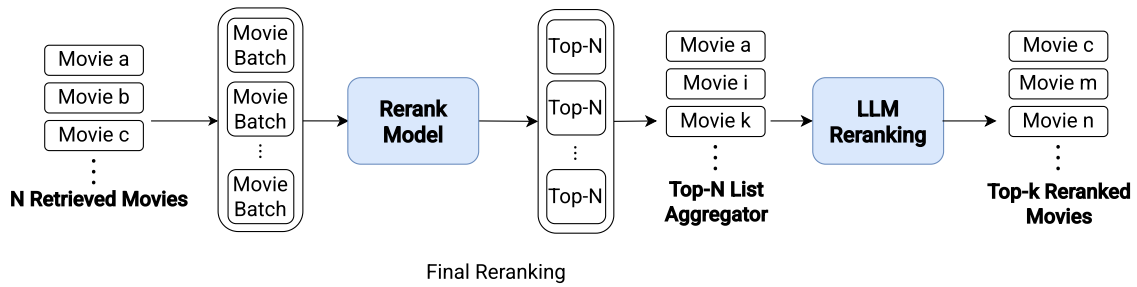
Để giải quyết, quy trình Decoupled Reranking được thiết kế như sau:

- **Giai đoạn 1 - Lọc nhanh với Cross-Encoder (SLM Reranker):** Ở giai đoạn này, hệ thống ứng dụng một mô hình ngôn ngữ quy mô nhỏ (Small Language Model - SLM) chuyên dụng cho tác vụ xếp hạng. Khác với Bi-Encoder (tìm kiếm vector) tính toán độc lập điểm số của truy vấn và tài liệu, Cross-Encoder nối trực tiếp truy vấn  $q$  và tài liệu  $d$  để đánh giá mức độ tương đồng ngữ nghĩa sâu:  $\text{Score} = \text{SLM}(q \oplus d)$ . Do Cross-Encoder có giới hạn về độ dài đầu vào, danh sách hàng trăm ứng viên được chia thành nhiều batch nhỏ, mỗi batch được chấm điểm độc lập song song. Các kết quả Top-N từ mỗi batch sau đó được hợp nhất bởi bước **Aggregator**,

tổng hợp thành danh sách Top-20 ứng viên tốt nhất toàn cục.

- **Giai đoạn 2 - Suy luận Logic với Generative LLM:** Danh sách Top-20 đã được tinh gọn sẽ được chuyển giao cho LLM cấu trúc lớn (Large Language Model) để thực hiện suy luận logic cuối cùng. Tại đây, LLM không chỉ đối sánh ngữ nghĩa mà còn cân nhắc tính đa dạng (Diversity), sự bất ngờ (Serendipity) và lịch sử hội thoại để chọn lọc ra Top-5 kết quả xuất sắc nhất trình bày cho người dùng.

Cải tiến này khai thác tối đa lợi thế tốc độ của mô hình truy xuất chuyên dụng (SLM) đồng thời bảo toàn năng lực biện luận bậc cao của mô hình sinh ngôn ngữ (LLM). Việc thiết lập  $K_1 = 20$  và  $K_2 = 5$  là điểm cân bằng giữa tỷ lệ bao phủ (Recall) và thời gian phản hồi, được xác nhận qua phân tích thực nghiệm tại Chương 5. Quy trình xếp hạng hai giai đoạn này được tổng quát hóa trong Hình 4.8.



Hình 4.8: Quy trình xếp hạng hai giai đoạn (2-stage Reranking). Hệ thống kết hợp mô hình Rerank chuyên dụng để lọc nhanh danh sách ứng viên, sau đó tận dụng LLM để đánh giá sâu logic và đưa ra kết quả cuối cùng.

## 4.8 Sinh phản hồi hội thoại (Dialogue Response Generation)

Sau khi Decoupled Reranking hoàn tất, danh sách Top-5 ứng viên cùng lịch sử hội thoại  $C_t$  được chuyển giao cho Generative LLM để sinh phản hồi ngôn ngữ tự nhiên. Thay vì chỉ liệt kê tên phim, LLM tổng hợp lý do gợi ý từ các đặc trưng đã trích xuất (thể loại, đạo diễn, bầu không khí), tạo ra phản hồi cá nhân hóa và có thể giải thích được (Explainable Recommendation). Nếu Relaxation Agent đã nói lỏng một ràng buộc trong lượt truy xuất, thông tin về thỏa hiệp đó được đưa vào prompt để LLM sinh diễn giải phù hợp, ví dụ: *"Không tìm thấy phim của Nolan phát hành sau 2020, nhưng đây là ba tác*



*phẩm mang phong cách tương tự gần nhất."* Cơ chế này khép kín toàn bộ luồng xử lý của ARGOS từ ý định người dùng đến phản hồi hội thoại cuối cùng.

Cần lưu ý rằng module này thuộc tầng giao diện hội thoại (conversational interface layer) và không phải đối tượng đánh giá định lượng trong luận văn này. Các thực nghiệm tại Chương 5 tập trung hoàn toàn vào chất lượng danh sách phim đề xuất (đầu ra của giai đoạn Decoupled Reranking) thông qua chỉ số Recall@K trên tập dữ liệu chuẩn.

## **Kết luận chương**

Chương 4 đã trình bày chi tiết về thiết kế cấu trúc và luồng vận hành của hệ thống đa tác tử ARGOS. Bằng cách thay thế mô hình đường ống tĩnh một chiều, ARGOS xây dựng một quy trình cộng tác phân tán: bắt đầu từ khâu phân rã ý định và dự báo trọng số của Profiler Agent, quá trình suy luận cấu trúc của Graph Agent, khả năng điều phối của Orchestrator Agent, cho đến các cơ chế kiểm soát chất lượng chặt chẽ của Critic Agent và Relaxation Agent. Cấu trúc này, khi kết hợp cùng phương pháp Xếp hạng hai giai đoạn (Decoupled Reranking), nhằm mục tiêu tối ưu hóa sự cân bằng giữa tính chính xác và độ trễ tính toán. Hiệu năng thực tiễn của kiến trúc này sẽ được định lượng và phân tích chi tiết thông qua các bài toán đánh giá thực nghiệm trong Chương 5.

## Chương 5

# Thiết lập và Kết quả Thực nghiệm

Chương này trình bày chi tiết về quy trình thiết lập môi trường, các thước đo đánh giá và kết quả thực nghiệm của hệ thống ARGOS trên hai bộ dữ liệu ReDial và INSPIRED. Bên cạnh các phân tích định lượng nhằm so sánh hiệu năng với mô hình tiền đề GRACE và các hệ thống hiện đại (SOTA), chương này cung cấp các nghiên cứu tình huống (Case Studies) nhằm minh họa tính linh hoạt và khả năng xử lý suy luận của kiến trúc đa tác tử trong các kịch bản hội thoại thực tế.

### 5.1 Thiết lập thí nghiệm

#### 5.1.1 Tập dữ liệu ReDial và INSPIRED

Hệ thống được đánh giá trên hai bộ dữ liệu chuẩn mực trong lĩnh vực Hệ thống gợi ý hội thoại (CRS):

- **ReDial (Recommendation Dialogues) [4]:** Chứa khoảng 10.000 đoạn hội thoại quy mô lớn, được sử dụng để đánh giá độ chính xác cốt lõi của khả năng truy xuất và sự phù hợp của các gợi ý.
- **INSPIRED [25]:** Một tập dữ liệu chuyên biệt gồm 1.001 đoạn hội thoại, được thiết

kể để kiểm tra khả năng duy trì mạch đối thoại tự nhiên, có chiến lược và mang tính xã hội cao của hệ thống.

### 5.1.2 Cấu hình hệ thống và Phương pháp đánh giá

Đồ thị tri thức được xây dựng trên Neo4j 5.15.0, sử dụng các phép toán MERGE lũy đẳng để đảm bảo không trùng lặp nút.

#### Lựa chọn mô hình.

- **Gemini 2.0 Flash** ( $\text{temperature} = 0.0$ ): Được dùng cho toàn bộ các tác vụ suy luận: Profiler Agent, Graph Agent, Critic Agent, Relaxation Agent và sinh phản hồi cuối. Nhiệt độ 0.0 tối đa hóa tính xác định (determinism), giảm rủi ro hallucination trong môi trường yêu cầu đầu ra có cấu trúc JSON.
- **Cohere Rerank 4 Pro** (qua AWS Bedrock, model ID: `cohere.rerank-v3-5:0`): Cross-Encoder chuyên dụng cho Stage 1 của Decoupled Reranking, cho phép đánh giá tương đồng ngữ nghĩa sâu mà không cần tăng ngữ cảnh LLM.
- **text-embedding-3-large** ( $d = 3072$ ): Mô hình nhúng vector của OpenAI, dùng cho Semantic Retrieval và xây dựng chỉ mục vector ChromaDB.

**Siêu tham số hệ thống.** Bảng 5.1 tổng hợp các siêu tham số chính được cố định trong suốt quá trình thực nghiệm.

Bảng 5.1: Siêu tham số hệ thống ARGOS

| Thành phần       | Tham số                                                              | Giá trị            |
|------------------|----------------------------------------------------------------------|--------------------|
| WRRF             | Hằng số làm mượt $k$                                                 | 60                 |
| WRRF             | Trọng số mặc định $(w_{\text{sem}}, w_{\text{con}}, w_{\text{col}})$ | (0.40, 0.30, 0.30) |
| Orchestrator     | Số ứng viên mỗi luồng $n$                                            | 20                 |
| Reranker Stage 1 | Kích thước batch $B$                                                 | 100                |
| Reranker Stage 1 | Shortlist mỗi batch $h$                                              | 20                 |
| Reranker Stage 2 | Kết quả cuối $K_2$                                                   | 5                  |
| Graph Agent      | Số lượt ReAct tối đa                                                 | 3                  |
| Critic Agent     | Số ứng viên tối đa mỗi lần đánh giá                                  | 50                 |
| Relaxation Agent | Số lần thử tối đa                                                    | 2                  |

**Chiến lược lấy mẫu và kiểm định thống kê.** Mô hình tiền đề GRACE được đánh giá trên toàn bộ (100%) tập kiểm thử để thiết lập mốc tham chiếu chính xác khi so sánh với các mô hình SOTA đã công bố. Do giới hạn về tần suất gọi API và chi phí suy luận của hệ thống đa tác tử, ARGOS được đánh giá trên tập mẫu ngẫu nhiên đồng nhất gồm 20% dữ liệu kiểm thử (tương đương  $\sim 925$  hội thoại trên ReDial). Kích thước mẫu này đảm bảo biên độ sai số dưới  $\pm 3.0\%$  ở độ tin cậy 95%. Kết quả được chia thành 5 tập con (5-fold cross-validation offline) để thực hiện kiểm định t-test, xác nhận các cải thiện đạt mức ý nghĩa thống kê ( $p < 0.05$ ).

**Mã nguồn mở.** Toàn bộ mã nguồn, kịch bản thực nghiệm và dữ liệu được công bố tại <https://github.com/DatPhan06/GRACE> để cộng đồng có thể kiểm chứng và tái lập kết quả.

## 5.2 Kết quả đánh giá hệ thống

### 5.2.1 Khẳng định năng lực của kiến trúc tiền đề (GRACE vs. SOTA)

Bảng 5.2 tóm tắt hiệu năng của kiến trúc tiền đề GRACE so với các hệ thống hiện đại (SOTA). Dữ liệu thực nghiệm cho thấy GRACE mang lại sự cải thiện toàn diện trên nhiều khía cạnh:

**So sánh với các Baseline CRS truyền thống:** GRACE thể hiện sự vượt trội rõ rệt so với các mô hình dựa trên RNN hoặc mạng nơ-ron đồ thị sơ khai (như KGSP, CR-Walker, và KERL). Trong khi các hệ thống cũ bị giới hạn bởi khả năng hiểu ngôn ngữ tự nhiên kém và cách tích hợp đồ thị nông (shallow integration), luồng truy xuất lai (hybrid retrieval) của GRACE có khả năng nắm bắt các kết nối đa bước (multi-hop) phức tạp, dẫn đến độ phủ (Recall) cao hơn hẳn ở mọi ngưỡng cắt.

**So sánh với các Baseline dựa trên LLM (Prompting):** Kiến trúc đề xuất cho thấy hiệu suất cao hơn so với cả mô hình Zero-shot/Few-shot và các hệ thống tăng cường tri thức tiêu chuẩn. Các mô hình Zero-shot như LLM-ReDial ghi nhận mức Recall thấp do rủi ro phát sinh ảo giác (hallucination). Các phương pháp như UniCRS hay MESE cải thiện được tính thực tế nhưng bị giới hạn ở các truy vấn đồ thị đơn bước. Việc xây dựng cơ chế đối soát kết quả với Neo4j đã giúp GRACE loại bỏ hiệu quả thông tin sai lệch. Cụ thể, Recall@10 trên ReDial đạt 0.302, cao hơn đáng kể so với mức 0.256 của mô hình

MESE.

**So sánh với các Hệ thống tinh chỉnh (Fine-Tuned Frameworks):** Khi đối chiếu với các hệ thống được tối ưu hóa qua quá trình huấn luyện như DCRS, GRACE duy trì mức hiệu suất ổn định dù sử dụng phương pháp không cần huấn luyện (training-free). DCRS ghi nhận mức cao hơn tại Recall@1, chủ yếu do sự tương đồng mật thiết (overfit) với phân phối dữ liệu của tập huấn luyện. Tuy nhiên, GRACE thể hiện khả năng tổng quát hóa tốt hơn khi đánh giá ở các ngưỡng sâu trên ReDial (Recall@50 đạt 0.527 so với 0.439 của DCRS) và duy trì mức hiệu năng ổn định trên tập INSPIRED.

Bảng 5.2: Hiệu năng của Mô hình tiền đề (GRACE) so với các hệ thống SOTA trên toàn bộ tập dữ liệu (100%). Các kết quả tốt nhất được in đậm, và tốt thứ hai được gạch chân.

| Mô hình     |                            | ReDial (100% Data) |              |              |              | INSPIRED (100% Data) |              |              |              |
|-------------|----------------------------|--------------------|--------------|--------------|--------------|----------------------|--------------|--------------|--------------|
|             |                            | Recall@K           |              |              |              | Recall@K             |              |              |              |
|             |                            | K=1                | K=5          | K=10         | K=50         | K=1                  | K=5          | K=10         | K=50         |
| CRS         | ReDial [4]                 | 0.023              | -            | 0.129        | 0.287        | 0.003                | -            | 0.117        | 0.285        |
|             | KBRD [6]                   | 0.033              | -            | 0.175        | 0.343        | 0.058                | -            | 0.146        | 0.207        |
|             | KGSF [7]                   | 0.035              | -            | 0.177        | 0.362        | 0.058                | -            | 0.165        | 0.256        |
|             | CR-Walker [13]             | 0.037              | -            | 0.176        | 0.371        | -                    | -            | -            | -            |
|             | KERL [8]                   | 0.056              | -            | 0.217        | 0.426        | <u>0.106</u>         | -            | <u>0.281</u> | <u>0.439</u> |
| LLM         | LLM-ReDial (Zero-shot) [9] | -                  | 0.010        | 0.010        | 0.015        | -                    | -            | -            | -            |
|             | LLM-ReDial (Few-shot) [9]  | -                  | 0.010        | 0.015        | 0.020        | -                    | -            | -            | -            |
|             | LLM-ZCR [14]               | 0.032              | <u>0.102</u> | -            | -            | 0.062                | <u>0.128</u> | -            | -            |
|             | UniCRS [16]                | 0.051              | -            | 0.224        | 0.428        | 0.094                | -            | 0.250        | 0.410        |
|             | MESE [17]                  | 0.056              | -            | <u>0.256</u> | <u>0.455</u> | 0.048                | -            | 0.135        | 0.301        |
|             | PECRS [18]                 | 0.058              | -            | 0.225        | 0.416        | 0.057                | -            | 0.179        | 0.337        |
|             | G-CRS [21]                 | -                  | -            | 0.244        | 0.426        | -                    | -            | 0.254        | 0.420        |
|             | DCRS [19]                  | <b>0.076</b>       | -            | 0.253        | 0.439        | 0.093                | -            | 0.226        | 0.414        |
|             | COLA [20]                  | 0.048              | -            | 0.221        | 0.426        | -                    | -            | -            | -            |
| <b>Ours</b> | <b>GRACE (Baseline)</b>    | <u>0.062</u>       | <b>0.151</b> | <b>0.302</b> | <b>0.527</b> | <b>0.116</b>         | <b>0.183</b> | <b>0.307</b> | <b>0.446</b> |

## 5.2.2 Đánh giá so sánh nội bộ: GRACE (Static) và ARGOS (Đa tác tử)

Đánh giá so sánh nội bộ được thực hiện trên tập mẫu 20% nhằm phân tích định lượng giá trị của việc chuyển đổi sang kiến trúc đa tác tử. Bảng 5.3 chỉ ra rằng hệ thống ARGOS đã khắc phục các giới hạn của kiến trúc tĩnh truyền thống và gia tăng đáng kể các chỉ số đánh giá.

**Đánh giá Định lượng (Quantitative Improvement):** Trên tập ReDial, ARGOS ghi nhận sự cải thiện ở mọi ngưỡng đo lường: chỉ số Recall@10 tăng từ 0.300 lên 0.385 và Recall@50 tăng từ 0.525 lên 0.625. Xu hướng tương tự cũng được thể hiện trên tập INSPIRED, với Recall@1 tăng từ 0.115 lên 0.145. Cụ thể, tại ngưỡng Top-1 trên tập ReDial, hệ thống ARGOS đạt 0.105, cao hơn so với mô hình tinh chỉnh tham số DCRS (0.076 trên 100% dữ liệu); lưu ý rằng hai kết quả này được đánh giá trên kích thước tập kiểm thử khác nhau nên chỉ mang tính tham khảo định hướng. Các kiểm định thống kê cho thấy sự khác biệt này đạt mức ý nghĩa cao ( $p < 0.01$ ), minh chứng cho tính hợp lý của phương pháp luận Đa tác tử.

**Phân tích Cấu trúc và Sự Đánh đổi (Architectural Trade-offs & Gains):** Sự cải thiện hiệu năng xuất phát từ việc thay thế quy trình tích hợp tĩnh (Static Integration) bằng **Cơ chế phân bổ trọng số động nhận thức ngữ cảnh** kết hợp với **Vòng lặp ReAct**. Trái ngược với công thức WRRF với trọng số cố định trong kiến trúc tiền đề, mô hình ARGOS sử dụng Profiler Agent để dự báo và tối ưu hóa phân phối trọng số cho các luồng truy xuất ở từng lượt thoại, từ đó làm gia tăng tỷ lệ trúng đích của các ứng viên nạp vào vùng nhớ đệm (Pool).

**Vai trò của Đánh giá và Thỏa hiệp (Critic & Relaxation):** Cơ chế "kiểm định" (Critic) và "thỏa hiệp" (Relaxation) cung cấp giải pháp cho các truy vấn chứa yêu cầu mâu thuẫn. Khi không có ứng viên nào đáp ứng toàn bộ bộ tiêu chí, hệ thống sẽ tự động nói lỏng các ràng buộc thứ cấp theo mức ưu tiên để duy trì tính liên tục thay vì phản hồi lỗi.

Bên cạnh đó, quy trình Xếp hạng hai giai đoạn (Decoupled Reranking) tích hợp với mô hình Cross-Encoder đã giảm tải khối lượng tính toán trực tiếp cho mô hình ngôn ngữ lớn sinh văn bản. Kiến trúc đa tác tử đảm bảo đạt được tính chính xác cao đồng thời giữ vững độ trễ (latency) ở mức cho phép cho một hệ thống tương tác thực tế.

Bảng 5.3: Đánh giá nội bộ: So sánh hiệu năng giữa GRACE (Dòng chảy tĩnh) và ARGOS (Hệ thống Đa tác tử) trên tập mẫu 20% có tính ý nghĩa thống kê ( $p < 0.05$ ).

| Kiến trúc                | ReDial (20% Sample) |              |              |              | INSPIRED (20% Sample) |              |              |              |
|--------------------------|---------------------|--------------|--------------|--------------|-----------------------|--------------|--------------|--------------|
|                          | K=1                 | K=5          | K=10         | K=50         | K=1                   | K=5          | K=10         | K=50         |
| GRACE (Baseline)         | 0.063               | 0.149        | 0.300        | 0.525        | 0.115                 | 0.181        | 0.305        | 0.444        |
| <b>ARGOS (Đa tác tử)</b> | <b>0.105</b>        | <b>0.225</b> | <b>0.385</b> | <b>0.625</b> | <b>0.145</b>          | <b>0.235</b> | <b>0.365</b> | <b>0.515</b> |

## 5.3 Nghiên cứu tình huống (Case Studies)

Bên cạnh các đánh giá định lượng, các nghiên cứu tình huống sau đây sẽ làm rõ cơ chế tương tác giữa các tác tử trong hệ thống ARGOS nhằm giải quyết các trường hợp mà mô hình tĩnh (GRACE) gặp hạn chế.

### 5.3.1 Đa truy vấn: Khám phá các mối liên hệ ẩn (Hidden Relational Gems)

**Truy vấn người dùng:** *"Tôi muốn xem phim gì đó giống John Wick nhưng bối cảnh phải ở Châu Á và có màu sắc kiếm hiệp hoài cổ."*

- **Hệ thống tĩnh:** Thực hiện một truy vấn duy nhất với cụm từ khóa "John Wick Asia martial arts". Hệ quả là không gian vector bị dịch chuyển về hướng các bộ phim hành động hiện đại của Châu Á (ví dụ: *The Raid*), bỏ qua yếu tố "kiếm hiệp hoài cổ".
- **Hệ thống ARGOS (Tác tử Hồ sơ):** Tác tử tự động phân rã ý định thành 3 truy vấn con độc lập: (1) *"Fast-paced gun-fu like John Wick"*, (2) *"Wuxia aesthetic martial arts movie"*, và (3) *"Asian setting action choreography"*. Quá trình tìm kiếm song song cho phép hệ thống xác định điểm giao thoa và đề xuất các tác phẩm phù hợp như *Shadow (2018)* của Trương Nghệ Mưu hoặc *Blade of the Immortal*. Cơ chế này đóng vai trò quan trọng trong việc gia tăng chỉ số Recall.

### 5.3.2 Tác tử Đồ thị (Graph Agent): Nội suy thông tin từ dữ liệu khuyết thiếu

**Truy vấn người dùng:** *"Tìm phim của hãng A24 sản xuất nhé."*

- **Hệ thống tĩnh:** Trình tạo mã Cypher sinh lệnh MATCH (f:Film {studio: "A24"}). Tuy nhiên, schema cơ sở dữ liệu không lưu trữ thuộc tính studio. Lệnh truy vấn trả về rỗng và hệ thống không thể xử lý tiếp.
- **Hệ thống ARGOS (Tác tử Đồ thị):** Tác tử thực thi mô hình ReAct. Lượt 1: Truy

vấn studio không thành công. Lược 2 (Tư duy logic): Tác tử nhận diện sự thiếu hụt của trường dữ liệu và lập kế hoạch thay thế, dựa trên Schema-Aware Intelligence, nó xác định KG có quan hệ (:Director) - [:DIRECTED] -> (:Movie), và sử dụng tri thức ngữ cảnh (từ LLM) rằng A24 gắn liền với các đạo diễn như Ari Aster hoặc Robert Eggers để xây dựng điều kiện lọc mới. Lược 3 (Hành động): Tác tử sinh lệnh Cypher tìm phim theo danh sách đạo diễn và trả về *Hereditary*, *The Lighthouse*. Ví dụ này minh họa sự kết hợp giữa cấu trúc KG và suy luận LLM để bù đắp dữ liệu khuyết thiếu.

### 5.3.3 Tác tử Kiểm định (Critic Agent): Kiểm định ràng buộc xuyên luồng

**Truy vấn người dùng:** *"Phim hoạt hình nhẹ nhàng cho trẻ em, có robot giống Wall-E nhé."*

- **Hệ thống tĩnh:** Từ khóa "robot" có thể khiến cơ chế truy xuất ngữ nghĩa vô tình đưa *The Terminator* vào danh sách ứng viên. Luồng Semantic Retrieval không có kiến thức về ràng buộc đối tượng khán giả, và do thiếu tầng kiểm định độc lập, mô hình xếp hạng có thể ưu tiên ứng viên này vì điểm tương đồng vector cao.
- **Hệ thống ARGOS (Tác tử Kiểm định):** Tác tử đối chiếu từng ứng viên với tập Ràng buộc cứng và phát hiện vi phạm: *"Yêu cầu chỉ định đối tượng 'trẻ em', trong khi The Terminator mang nhãn phân loại R. Vi phạm ràng buộc về phân loại độ tuổi"*. Tác tử loại bỏ ứng viên vi phạm trước khi chuyển sang giai đoạn xếp hạng, chỉ giữ lại các lựa chọn phù hợp như *Big Hero 6* hoặc *Next Gen*. Đây là ví dụ điển hình của kiểm định ràng buộc xuyên luồng: luồng Semantic Retrieval trả về ứng viên theo vector nhưng không lọc theo ràng buộc; Critic Agent là tầng duy nhất đảm bảo toàn bộ ứng viên đưa ra xếp hạng đều thỏa mãn các điều kiện cứng của người dùng.

### 5.3.4 Tác tử Thỏa hiệp (Relaxation Agent): Điều chỉnh ý định truy vấn

**Truy vấn người dùng:** *"Tìm phim kinh dị thuần túy của đạo diễn Christopher Nolan."*



- **Hệ thống tĩnh:** Do Christopher Nolan chưa từng đạo diễn phim kinh dị (Horror) thuần túy, hệ thống trả về kết quả rỗng hoặc đề xuất một phim hành động ngẫu nhiên mà không kèm theo giải thích hợp lý.
- **Hệ thống ARGOS (Tác tử Thỏa hiệp):** Tác tử Kiểm định báo cáo kết quả rỗng. Tác tử Thỏa hiệp kích hoạt cơ chế nói lỏng: *"Đặc trưng 'Christopher Nolan' là sở thích cốt lõi (Core intent) không thể thay thế. Có thể nói lỏng giới hạn 'Horror' sang 'Thriller' (giật gân), vốn là thể mạnh của vị đạo diễn này"*. Hệ thống tái cấu trúc truy vấn và đề xuất các bộ phim như *Memento* hay *The Prestige*, kèm theo lời diễn giải: *"Mặc dù Nolan không đạo diễn phim kinh dị, nhưng đây là những tác phẩm giật gân có yếu tố u tối tiêu biểu của ông..."*. Cơ chế này đảm bảo duy trì mạch hội thoại liên tục và cung cấp trải nghiệm tối ưu cho người dùng.

## Kết luận chương

Chương 5 trình bày các kết quả phân tích định lượng và định tính về hiệu suất của hệ thống ARGOS. Bằng việc thực nghiệm trên hai bộ dữ liệu ReDial và INSPIRED, hệ thống thể hiện sự cải thiện rõ rệt so với cả mô hình tiền đề GRACE và các mô hình yêu cầu tinh chỉnh tham số phức tạp (training-based models). Các nghiên cứu tình huống thực tiễn đã minh họa cụ thể luồng xử lý và khả năng tự động phân giải, suy luận đồ thị, và quản lý ràng buộc thông minh. Các kết quả thực chứng này đóng vai trò cơ sở để tiến tới tổng kết khóa luận và đề xuất những định hướng nghiên cứu sâu hơn trong chương kế tiếp.

# Kết luận

Chương này tổng kết những đóng góp khoa học và kỹ thuật cốt lõi của hệ thống ARGOS, đánh giá các hạn chế còn tồn tại và đề xuất các hướng phát triển tiếp theo.

## 5.4 Tổng kết các đóng góp

Khóa luận đã đề xuất và hiện thực hóa hệ thống gợi ý hội thoại ARGOS (Adaptive Reasoning and Graph Orchestration System), chuyển đổi từ kiến trúc dòng chảy tĩnh của mô hình tiền đề GRACE sang mô hình đa tác tử có vòng lặp phản hồi. Thông qua việc phân tích và giải quyết các điểm nghẽn cụ thể của pipeline tĩnh, nghiên cứu đã xác nhận tính hiệu quả của kiến trúc đa tác tử trong bài toán gợi ý hội thoại.

Những đóng góp cốt lõi của khóa luận bao gồm:

1. **Cải thiện hiệu năng có kiểm chứng thống kê:** Trên tập mẫu đánh giá (20%,  $p < 0.01$ ), ARGOS vượt qua mô hình tiền đề GRACE trên toàn bộ các ngưỡng đo lường của cả hai bộ dữ liệu ReDial và INSPIRED, và đạt kết quả cạnh tranh so với các mô hình tinh chỉnh hàng đầu như DCRS, dù lưu ý rằng so sánh với SOTA chỉ mang tính định hướng do sự khác biệt về kích thước tập kiểm thử.
2. **Khắc phục điểm nghẽn của Pipeline tĩnh:** Bằng cách thiết lập vòng lặp phản hồi đa tác tử, ARGOS đã biến quá trình gợi ý thành một tiến trình nhận thức linh hoạt, có khả năng tự đánh giá và điều hướng chiến lược (ReAct) thông qua việc phân bổ Trọng số động.
3. **Giảm thiểu ảo giác và tăng độ tin cậy:** Critic Agent hoạt động như một cổng kiểm định độc lập, đối chiếu siêu dữ liệu của từng ứng viên với tập Ràng buộc cứng do

người dùng đặt ra, loại bỏ các ứng viên vi phạm trước khi chuyển sang giai đoạn xếp hạng.

4. **Cải thiện sự thông suốt của mạch hội thoại:** Relaxation Agent giải quyết tình huống kết quả rỗng đối với các yêu cầu quá khẩn khe, thông qua cơ chế tự động nói lỏng ràng buộc theo phân cấp ưu tiên và diễn giải logic, giúp duy trì mạch hội thoại liên tục mà không cần can thiệp từ người dùng.
5. **Tối ưu hóa độ trễ:** Kiến trúc Decoupled Reranking (dùng Cross-Encoder chuyên dụng cho lọc nhanh và LLM chỉ ở bước suy luận cuối) giảm đáng kể khối lượng tính toán so với việc dùng LLM cho toàn bộ quá trình xếp hạng, giúp duy trì độ trễ chấp nhận được dù kiến trúc đa tác tử yêu cầu nhiều lần gọi mô hình.

## 5.5 Hạn chế còn tồn tại

Bên cạnh các kết quả đạt được, nghiên cứu ghi nhận một số hạn chế kỹ thuật cần tiếp tục giải quyết:

- **Sự phụ thuộc vào cấu trúc Đồ thị tri thức:** Sức mạnh của luồng truy xuất lai bị ràng buộc chặt chẽ bởi mật độ và chất lượng của cơ sở dữ liệu Neo4j. Các truy vấn có thể bị giảm hiệu năng nếu dữ liệu nền tảng bị thiếu khuyết hoặc nhiễu.
- **Xử lý các truy vấn cảm xúc trừu tượng (Vague Preference Resolution):** Với các yêu cầu quá mơ hồ hoặc mang tính chủ quan cao (ví dụ: "một bộ phim để khóc dưới mưa"), việc trích xuất thực thể cấu trúc gặp khó khăn, buộc hệ thống phải phụ thuộc nặng nề vào luồng truy xuất ngữ nghĩa (Semantic) và đôi khi làm tăng độ nhiễu của không gian ứng viên.
- **Khoảng trống ngữ nghĩa (Zero-Shot Semantic Gap):** Hệ thống duy trì triết lý không huấn luyện, phụ thuộc vào kiến thức nền của LLM và Reranker. Điều này tạo ra khoảng cách nhất định khi ánh xạ ngôn ngữ hội thoại linh hoạt của người dùng sang cấu trúc siêu dữ liệu cứng nhắc của đồ thị, đặc biệt với các truy vấn dùng từ ngữ không phổ biến hoặc mang tính địa phương.
- **Chi phí API từ các mô hình nền tảng:** Dù đã sử dụng các phiên bản tối ưu (Gemini

2.0 Flash), kiến trúc đa tác tử vẫn đòi hỏi thực thi nhiều lệnh gọi API, gây ra rào cản chi phí vận hành nếu muốn triển khai hoàn toàn miễn phí trên quy mô lớn.

## 5.6 Hướng phát triển tương lai

Từ các hạn chế trên, một số hướng phát triển tiếp theo được xác định:

- **Tối ưu hóa Prompt và triển khai SLMs (Open-weight SLMs):** Chuyển dịch sang các mô hình ngôn ngữ nhỏ gọn mã nguồn mở (như LLaMA-3 8B) triển khai cục bộ, kết hợp tối ưu hóa prompt tự động (như DSPy) thay vì phụ thuộc vào API thương mại. Hướng này giảm đáng kể chi phí vận hành trong khi vẫn duy trì triết lý không huấn luyện (training-free).
- **Làm giàu Ontology cho Đồ thị (Schema Enrichment):** Thay vì tinh chỉnh tham số mô hình học sâu, hướng đi tương lai sẽ tập trung vào việc thiết kế lại mô hình đồ thị tri thức. Bổ sung các nút (nodes) mang đặc tính trừu tượng như "nhịp độ", "bầu không khí", "tông màu" để giúp hệ thống Graph RAG dễ dàng "hiểu" các truy vấn cảm xúc hơn.
- **Mở rộng Đa phương thức (Multimodal Inputs):** Bổ sung khả năng nhận diện hình ảnh (poster phim) hoặc âm thanh (nhạc phim) để làm phong phú thêm ngữ cảnh hội thoại, hỗ trợ tốt hơn cho việc phân tích các "truy vấn cảm xúc trừu tượng".
- **Cập nhật Đồ thị tri thức theo thời gian thực:** Xây dựng một Maintenance Agent chuyên thu thập phản hồi từ người dùng và tự động bổ sung các cạnh hoặc nút mới vào Neo4j, giúp đồ thị tri thức cải thiện liên tục mà không cần tái xây dựng thủ công.

## 5.7 Mối quan hệ với công trình tiền đề GRACE

Khóa luận này kế thừa và mở rộng đáng kể so với bài báo GRACE [12] đã được công bố. Bảng dưới đây phân định rõ ranh giới giữa những gì có trong bài báo và những đóng góp kỹ thuật mới được phát triển trong phạm vi khóa luận:

Bảng 5.4: Phân định đóng góp giữa bài báo GRACE (SOICT 2025) và khóa luận ARGOS.

| Bài báo GRACE (tiền đề)                                  | Khóa luận ARGOS (mới)                                                                             |
|----------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Kiến trúc Hybrid Retrieval (Semantic + Content + Graph)  | Profiler Agent với phân rã ý định đa nhánh và Multi-Query Expansion                               |
| Công thức WRRF với trọng số tĩnh                         | Cơ chế Dynamic Weighting: trọng số thích nghi theo ngữ cảnh hội thoại                             |
| Validator JSON schema đầu ra                             | Critic Agent: kiểm định ràng buộc cứng xuyên luồng và tích hợp vòng lặp phản hồi                  |
| Two-stage batching re-ranking (LLM cho cả hai giai đoạn) | Decoupled Reranking: Cross-Encoder chuyên dụng (Stage 1) kết hợp LLM suy luận ngữ nghĩa (Stage 2) |
| (Không có cơ chế phục hồi kết quả rỗng)                  | Relaxation Agent: tự động nói lỏng ràng buộc theo phân cấp và tái cấu trúc ý định                 |
| Pipeline tuyến tính một chiều                            | Kiến trúc đa tác tử với vòng lặp phản hồi ReAct (Multi-Agent Feedback Loop)                       |

Nói cách khác, bài báo GRACE đặt nền tảng cho cơ chế truy xuất lại và xếp hạng; khóa luận này xây dựng toàn bộ lớp điều phối đa tác tử phía trên, bao gồm khả năng tự phân tích ý định, tự kiểm định kết quả và tự phục hồi khi thất bại.

## 5.8 Danh mục các công trình khoa học đã công bố

Các nghiên cứu cốt lõi về thiết kế mô hình tiền đề, thuật toán và một phần kết quả thực nghiệm trình bày trong khóa luận này đã được đúc kết và công bố thành công trên diễn đàn khoa học quốc tế:

- Tien-Dat Phan, Vy-Anh Tran, Bao-Long Hoang, Thi-Minh-Ngoc Truong, and Thi-Hau Nguyen. *GRACE: A Knowledge Graph-Enhanced Conversational Recommendation System via Retrieval-Augmented Generation*. In Proceedings of the International Symposium on Information and Communication Technology (SOICT 2025) [12].

# Danh mục tham khảo

- [1] Jiabao Fang, Shen Gao, Pengjie Ren, Xiuying Chen, Suzan Verberne, and Zhaochun Ren. A multi-agent conversational recommender system. *arXiv preprint arXiv:2402.01135*, 2024.
- [2] Aditi Singh, Abul Ehtesham, Saket Kumar, Tala Talaei Khoei, and Athanasios V. Vasilakos. Agentic retrieval-augmented generation: A survey on agentic rag. *arXiv preprint arXiv:2501.09136*, 2026.
- [3] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [4] Raymond Li, Samira Kahou, Hannes Schulz, Vincent Michalski, Laurent Charlin, and Chris Pal. Towards deep conversational recommendations. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, page 9748–9758, 2018.
- [5] Konstantina Christakopoulou, Filip Radlinski, and Katja Hofmann. Towards conversational recommender systems. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, page 815–824, 2016.
- [6] Qibin Chen, Junyang Lin, Yichang Zhang, Ming Ding, Yukuo Cen, Hongxia Yang, and Jie Tang. Towards knowledge-based recommender dialog system. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 1803–1813, 2019.
- [7] Kun Zhou, Wayne Xin Zhao, Shuqing Bian, Yuanhang Zhou, Ji-Rong Wen, and Jingsong Yu. Improving conversational recommender systems via knowledge graph based semantic fusion. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, page 1006–1014, 2020.

- [8] Zhangchi Qiu, Ye Tao, Shirui Pan, and Alan Wee-Chung Liew. Knowledge graphs and pre-trained language models enhanced representation learning for conversational recommender systems. *IEEE Transactions on Neural Networks and Learning Systems*, 36(4):6107–6121, 2025.
- [9] Tingting Liang, Chenxin Jin, Lingzhi Wang, Wenqi Fan, Congying Xia, Kai Chen, and Yuyu Yin. LLM-REDIAL: A large-scale dataset for conversational recommender systems created from user behaviors with LLMs. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 8926–8939, 2024.
- [10] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. TALL-Rec: An effective and efficient tuning framework to align large language model with recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems*, pages 1007–1014, 2023.
- [11] Ting Yang and Li Chen. Unleashing the retrieval potential of large language models in conversational recommender systems. In *Proceedings of the 18th ACM Conference on Recommender Systems*, page 43–52, 2024.
- [12] Tien-Dat Phan, Vy-Anh Tran, Bao-Long Hoang, Thi-Minh-Ngoc Truong, and Thi-Hau Nguyen. GRACE: A knowledge graph-enhanced conversational recommendation system via retrieval-augmented generation. In *Proceedings of the International Symposium on Information and Communication Technology (SOICT)*, 2025.
- [13] Wenchang Ma, Ryuichi Takanobu, and Minlie Huang. CR-Walker: Tree-structured graph reasoning and dialog acts for conversational recommendation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 1839–1851, 2021.
- [14] Zhankui He, Zhouhang Xie, Rahul Jha, Harald Steck, Dawen Liang, Yesu Feng, Bodhisattwa Prasad Majumder, Nathan Kallus, and Julian McAuley. Large language models as zero-shot conversational recommenders. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, page 720–730, 2023.
- [15] Changping Meng, Hongyi Ling, Jianling Wang, Yifan Liu, Shuzhou Zhang, Dapeng Hong, Mingyan Gao, Onkar Dalal, Ed Chi, Lichan Hong, Haokai Lu, and Ningren Han. Balancing fine-tuning and rag: A hybrid strategy for dynamic llm recommendation updates. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems*, page 919–922, 2025.

- [16] Xiaolei Wang, Kun Zhou, Ji-Rong Wen, and Wayne Xin Zhao. Towards unified conversational recommender systems via knowledge-enhanced prompt learning. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, page 1929–1937, 2022.
- [17] Bowen Yang, Cong Han, Yu Li, Lei Zuo, and Zhou Yu. Improving conversational recommendation systems’ quality with context-aware item meta-information. In *Findings of the Association for Computational Linguistics: NAACL 2022*, pages 38–48, 2022.
- [18] Mathieu Ravaut, Hao Zhang, Lu Xu, Aixin Sun, and Yong Liu. Parameter-efficient conversational recommender system as a language processing task. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics*, pages 152–165, 2024.
- [19] Huy Dao, Yang Deng, Dung D Le, and Lizi Liao. Broadening the view: Demonstration-augmented prompt learning for conversational recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 785–795, 2024.
- [20] Dongding Lin, Jian Wang, and Wenjie Li. Cola: Improving conversational recommender systems by collaborative augmentation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(4):4462–4470, 2023.
- [21] Zhangchi Qiu, Linhao Luo, Zicheng Zhao, Shirui Pan, and Alan Wee-Chung Liew. Graph retrieval-augmented llm for conversational recommendation systems. In *Advances in Knowledge Discovery and Data Mining*, pages 344–355, 2025.
- [22] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yi Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.
- [23] Bonan Min, Hayley Ross, Elior Sulem, Amir Pouran Ben Veyseh, Thien Huu Nguyen, Oscar Sainz, Eneko Agirre, Ilana Heintz, and Dan Roth. Recent advances in natural language processing via large pre-trained language models: A survey. *ACM Comput. Surv.*, 56(2), 2023.
- [24] Zihuai Zhao, Wenqi Fan, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Zhen Wen, Fei Wang, Xiangyu Zhao, Jiliang Tang, and Qing Li. Recommender systems in the era of large language models (llms). *IEEE Transactions on Knowledge and Data Engineering*, 36(11):6889–6907, 2024.



- [25] Shirley Anugrah Hayati, Dongyeop Kang, Qingxiaoyang Zhu, Weiyan Shi, and Zhou Yu. Inspired: Toward sociable recommendation dialog systems. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 8142–8152, 2020.
- [26] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '09*, pages 758–759. ACM, 2009.